创 新 发 明 作 品 报 告

论文题目

基于声明式意图编译与多专家智能体协同的自动化影音设计平台

平台名称：Eimu ｜ www.eimu.dev

An Automated Audiovisual Design and Production Platform Based on Declarative Intent Compilation and Multi-Expert Agent Collaboration

参赛类别	科技创新 / 软件与智能系统
作者姓名	陈天泽
指导教师	曾剑辉
所在学校	濠江中学

二〇二六年九月

摘　要

近年来，多模态生成模型已经实现在数百秒内产出图像、视频、音频与三维内容，但面向实际生产的设计能力仍然稀缺：现有工具大多提供素材生成或流程执行服务，然而从模糊意图到可交付成品的完整设计过程仍然依赖于人类思考与决策。为此，本文设计并实现了一个自动化影音设计平台 Eimu。该平台以声明式意图编译与多专家（Multi-Expert）智能体协同为核心机制：智能体不直接执行生成任务，而是将自然语言意图编译为**声明式意图中间表示**（Declarative Intent Intermediate Representation，Intent IR）；确定性创作图编译器对中间表示进行全图类型校验、依赖分析与原子提交，生成**可执行创作图**；用户可在同一张图上进行可视化编辑，并通过**人工编辑优先机制**（Human-Edit Precedence）保证人工修改不被自动布局覆盖。平台另设七类专业角色、审美一致性资产体系与提示词规范化模块，支持图像、视频、音乐、语音和三维五类媒介的协同生产。

实测结果表明，平台可自动生成结构化剧本、角色设定、分镜、关键帧、原创配乐与剪辑时间线，并产出 64.5 秒完整短片、36.3 秒多镜头片段、65.7 秒原创配乐及三维转台动画。系统包含 174,218 行 TypeScript 代码、154 个后端接口、90 个 AI 可调用工具与 1,816 项自动化测试，覆盖半成品落盘、重复扣费、账目失真、人工修改被覆盖、无效作业占用并发槽五类失败模式。本文的主要贡献包括：（1）提出人机共笔的可执行创作图范式；（2）实现「智能体编译而非执行」的意图编译机制；（3）构建审美一致性资产化方法；（4）设计面向提示词规范化的自动改写模块；（5）在 Harness 层把注入检测、专家路由、模板选择与记忆入库等判定从语言模型移交给可标定的决策模型，并以批量提问、前置过滤、fail-open 退化与同契约兜底将其工程化。

关键词：混合主动交互（Mixed-Initiative Interaction）；多智能体协作（Multi-Agent Collaboration）；智能体工作流（Agentic Workflow）；工作流编排（Workflow Orchestration）；Harness 工程（Harness Engineering）；人在环上（Human-in-the-Loop）；可控生成（Controllable Generation）；计算美学（Computational Aesthetics）；风格一致性（Style Consistency）

术语说明：本报告的关键术语与对应的既有概念

术语	英文	对应的既有概念（以既有术语定义）
混合主动交互	Mixed-Initiative Interaction	人机交互的经典范式（Horvitz, CHI 1999）：交互双方均持有主动权，可随时发起、接管与交还控制权。它区别于两种极端——单向的「指令—执行」，以及无人值守的全自动执行。
可执行创作图	Executable Creative Graph	将两类既有结构合而为一：面向人的**创作画布**，与面向机器的**数据流依赖图**（dataflow graph／DAG，连线即依赖）。它与业界的「智能体工作流」（agentic workflow）同类，区别在于：工作流的流程由人预先定义，本图由智能体声明后**编译**而成。
意图编译	Intent Compilation	沿用编译器的经典分层：前端 → 中间表示（IR）→ 校验与优化 → 后端（执行引擎）。它对应软件工程的**规格驱动开发**（spec-driven development）——先声明规格、经校验、再落地；差别在于此处的规格本身就是一张**可执行的图**，而非一份文档。前端由语言模型承担，概率性因此被限制在「生成中间表示」这一层。
意图中间表示	Intent Intermediate Representation（Intent IR）	编译器 IR 在此处的对应物：不描述执行步骤，只描述目标结构；必须通过类型与依赖校验后方可提交。
人机共笔	Human–AI Co-Authorship	对应版本控制中的既有实践——**人工修改优先于自动格式化**（manual override precedence）：双方编辑同一份对象，人工编辑具有优先级，自动化不得覆盖。其时代坐标是：氛围编程（vibe coding）→ 智能体工程（agentic engineering，人充当监督者）→ **人机共笔**（人成为同一份作品上的共同作者）。

术语	英文	对应的既有概念（以既有术语定义）
审美一致性资产化	Aesthetic Consistency as an Asset	将设计规范（design tokens／style guide）的既有做法扩展到生成式内容：把角色、调性、风格与参考固化为一组可版本化、可复用、可交接的资产。

Abstract

Recent advances in multimodal generative models have made the synthesis of images, video, audio, and 3D content fast and accessible, yet the design capability required for actual production remains scarce. Existing tools focus on either material generation or workflow execution, and can hardly complete the full design process from a vague intention to a deliverable result. This paper presents **Eimu**, an automated audiovisual design platform built upon declarative intent compilation and multi-expert agent collaboration.

In this platform, the agent does not execute generation tasks directly; it compiles natural-language intent into a **declarative intent intermediate representation** (Intent IR), which a **deterministic creative-graph compiler** validates through full-graph type checking and dependency analysis, and then commits atomically into an **executable creative graph**. The user edits the same graph visually, while a **human-edit precedence** mechanism ensures that manual edits are never overwritten by automatic layout. The platform further provides seven professional design roles, an aesthetic-consistency asset system, and a prompt-normalization module, supporting collaborative production across five media types: image, video, music, speech, and 3D.

Experimental results show that the system automatically produces structured scripts, character sheets, storyboards, keyframes, original scores, and editing timelines, yielding in particular a 64.5-second short film, a 36.3-second multi-shot sequence, a 65.7-second original score, and a 3D turntable animation. The implementation comprises 174,218 lines of TypeScript, 154 backend endpoints, 90 agent-callable tools, and 1,816 automated tests, covering five classes of failure modes: partially committed graphs, duplicated charging, distorted billing records, overwritten manual edits, and admitted-but-doomed jobs.

The main contributions are: (1) the paradigm of the human–AI co-authored executable creative graph; (2) an intent-compilation mechanism in which the agent compiles rather than executes; (3) an assetization method for aesthetic consistency; and (4) an automatic rewriting module for prompt normalization.

Keywords: Mixed-Initiative Interaction; Multi-Agent Collaboration; Agentic Workflow; Workflow Orchestration; Harness Engineering; Human-in-the-Loop; Controllable Generation; Computational Aesthetics; Style Consistency

目 录

一、选题背景

1.1 生成能力的爆发与设计能力的稀缺

1.2 现有 AI 创作工具的三个痛点

二、选题意义

2.1 AI 承担设计角色

2.2 对不同使用者的影响

三、平台定位与总体设计

3.1 平台定位

3.2 三层架构（图 1）

3.3 基座选择与二次开发

3.4 许可与项目贡献

3.5 工程目标与验证方式

3.6 开发过程的自反式迭代

四、交互范式：人机共笔的可复用工作流

4.1 三种既有范式的优劣势

4.2 Agent Harness 中的约束

4.3 人在环中：可视化编辑与人工编辑优先

4.4 创作状态的迁移：从对话到工作流

4.5 工作流的构成：一套类型系统承载生成、资产与编排

五、核心能力

5.1 来自自然语言需求的专业资产生成

5.2 审美一致性：固定参考影音资产

5.3 提示词规范化：从口语表达到镜头语言

5.4 设计质量控制：明确检查规则，统一交付标准

5.5 多媒介内容生产：五类媒介统一制作，同一内容生成多个版本

5.6 生成结果存档：自动保存，方便下次使用

六、核心创新点

七、成果验证

八、与同类方案的对比

九、总结与展望

附录（规模与界面证据）

参考文献

一、选题背景

一七八〇年五月，约翰·亚当斯在给妻子的信中写道：我们这一代必须先研究政治与战争，下一代才能自由地研究数学、哲学、工程和商业，再下一代才有机会研究绘画、诗歌、音乐和建筑 [1]。这段话强调了各学科发展的条件需要逐层积累：先解决秩序与生存，再积累知识与技术，最后艺术才可能成为更多人的生活现实。近年来，大模型快速普及，思维和任务能力大幅进步，影片、图片、音乐生成模型的迭代让内容变得容易生产；但内容的美观性和可应用性，仍取决于设计判断、审美一致性和人与工具的配合方式。原文中第三层的学科不但有突破的潜力，更有突破的必要：**生成能力解决内容的产生，设计能力解决内容的产出与应用**。

1.1 生成能力的爆发与设计能力的稀缺

近年来图像与视频生成模型快速演进，获取单件高质量素材已经相对简便；困难主要集中在进入生产流程的时候：生成内容之间缺乏结构关系，难以形成统一的风格与叙事。

设计能力可概括为四个方面：判断决定内容取舍，组织决定镜头与节奏，一致性保证角色与调性稳定，翻译将模糊意图转化为可执行的专业描述。当前这些环节主要依赖人工完成，模型仅提供像素级生成能力。因此，创作者普遍采用批量生成与人工筛选的方式（一次生成一批、人工挑选一张、局部不满意则整批重做）；生成成本下降并未同步降低专业影音资产的生产门槛。

1.2 现有 AI 创作工具的三个痛点

表 1　现有 AI 创作工具的共性痛点

痛点	具体表现	对创作流程的实际影响
资产碎片化	AI生成的内容缺乏分镜、节奏、和情绪曲线设计，只有孤立的一张图或一段影片	用户只获得零散的影音素材
一致性不足	角色在不同的生成素材中形象不断变化；同一系列的产品更换尺寸后视觉调性不统一；难以保持不同媒介之间的一致性	难以形成系列、品牌，只能产出单项影音素材
过程难复现	参数与配方不随作品保存	好的生成效果难以通过Harness复现，也无法与团队或Agent共创、迭代

三个痛点的共同特征是：虽然工具提供生成能力，设计和审美却仍然要交给人类思考，人和智能体之间也没有高效的协同创作方式。

二、选题意义

2.1 AI承担设计角色

本系统的目标，是让 AI 在生成素材之上，进一步承担设计工作。目前影视行业的设计工作主要分为数个部分：分镜、叙事、情绪递进、音乐转场、成片尾帧、模拟三维场景的机位运动等。这些决策与设计过去全部由人完成；本平台首次成功驱动 AI 自主完成，并且经过约束来校验产出结果。例如，系统可完成结构设计（故事节拍与镜头清单）、听觉设计（情绪需求、歌词与成曲）、时间设计（剪辑时间线与时长控制）、空间设计（三维场景、机位与运动）与质量判定（分镜的确定性检验）；用户的角色从软件操作者转变为需求提出者与最终决策者。

2.2 对不同使用者的影响

个人创作者的主要障碍在于专业软件与提示词门槛；本平台将编剧、导演、作曲、剪辑与三维美术等角色交由系统承担，使用者只需提出需求并做最终决策。

中小企业与团队的主要障碍在于生产周期与外包成本；本平台提供主流的开放OpenAI格式接口，接入Agent能力无需改动既有架构。

对内容产业而言，审美标准一旦确定和可复现，内容生产就能脱离逐件手工的状态，按流水线组织起来。

三、平台定位与总体设计

3.1 平台定位

Eimu 的定位是通用设计平台，上位替代了原有的中转站或者大模型厂商提供的官方API封装网站模式。其核心产出是影音素材之上更进一步的设计结果；平台解决使用门槛和难点，使用者只需表达需求；平台的核心资产是Harness与后端赋予的审美能力与设计流程，包括一致性维护、质量控制、意图翻译以及人与智能体的协同机制。平台覆盖图像、视频、音乐、语音与三维五类媒介，并将其统一到同一张创作图中：不同媒介被整合到了一创作流程中的不同工序，共享角色、调性与配方。

第一章的三代行业形成了递进关系：第二层（算力与模型）的进步，使得第三层（艺术创造的普及）的驱动力更丰富。模型每换代一次，单位生成的成本就下降一次，这对本系统直接有利。本系统与仅调用模型的产品的区别，在于模型换代之后的结果：后者的价值取决于模型供给，随上游换代逐层更换；前者留下的是审美判断、一致性资产与协作方式这类设计能力。据此，系统在架构上把设计决策与生成供给分开管理，形成下文的三层架构。三层中，设计层与 AI 设计能力层承载系统自有的能力资产，生成供给层仅负责调用外部模型，各自职责见 3.2。

3.2 三层架构

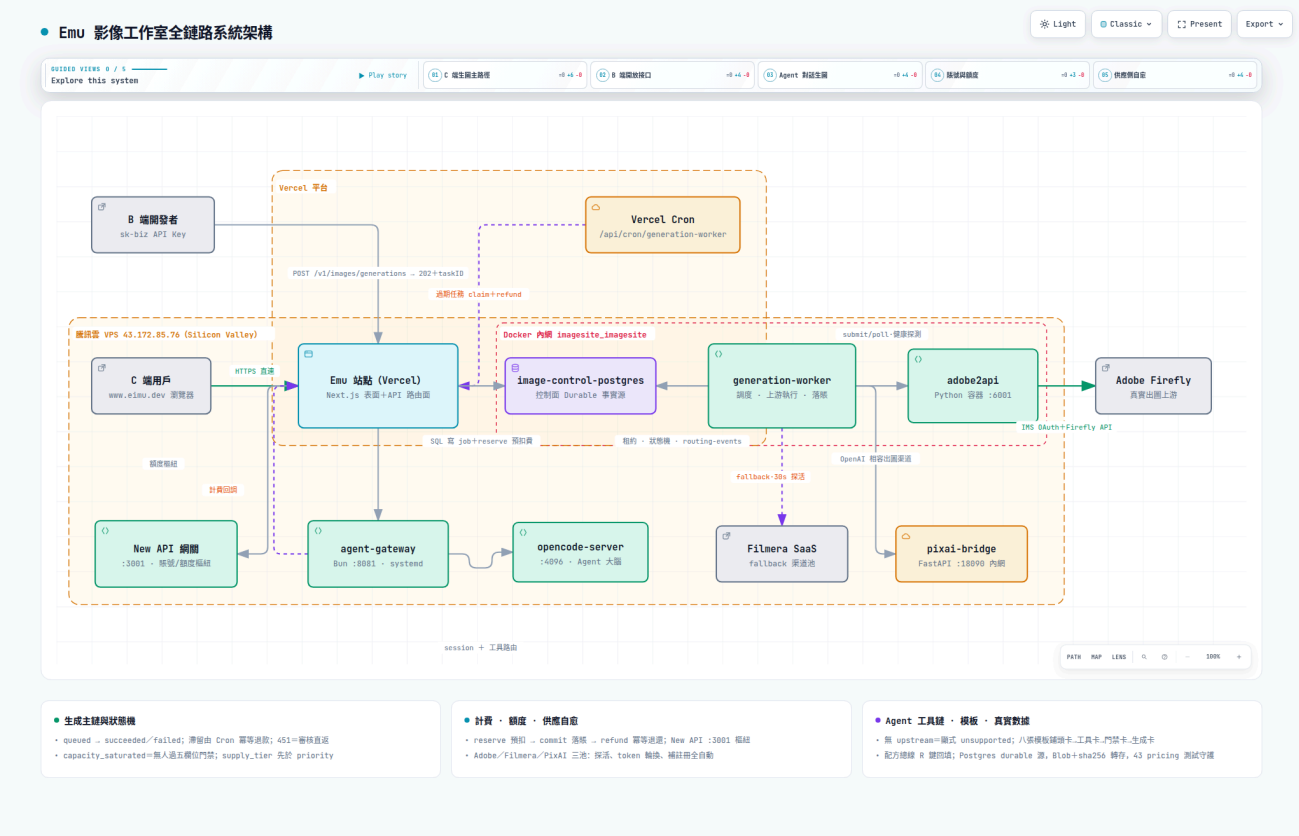


图 1　平台总体架构：设计层、AI 设计能力层、生成供给层

表 2　三层架构与各自职责

层次	主要作用	关键构成
设计层	前端人与 AI 共同工作的板块。人类提出需求、人与AI在同一张创作图上共创、分析、形成决策	需求入口（自然语言）、可执行创作图（包括创作画布与数据流依赖图）、公开案例与一键复刻入口、参数与费用展示
AI 设计能力层	把需求转化为设计决策并组织执行：分工、标准、验证方法论	七类专业角色、90 个可调用工具、617 份领域细分专业资料、交付规则与质量门禁
生成供给层	把设计意图经过分析、制作、打磨后，形成真实成果：调用、计费、交付	图像／视频／音乐／语音／三维五类能力、56 个模型、统一调度与消费记录

这种分层方法带来的直接优势是：**设计能力与生成能力解耦**。生成端可以通过new-api,在数小时内快速接入最新一代的生成模型，而平台的设计能力与审美保持一致：风格库、场景模板、配方模板、质量门禁与资产体系可以灵活地适应不同的影音模型。

3.3 基座选型与二次开发

本平台不重造智能体运行时与计费后端，而是在两个开源基座上独创深度集成：**执行引擎**选用开源多会话智能体框架 opencode（负责会话管理、工具调用循环与事件流），主要对以下四个方面进行了二次开发：一：搭载90 个自研设计工具以框架原生工具目录（`.opencode/tools/`），并另建实现审计表逐档核对每个工具背后真实存在的路由与处理函数——功能宣告必须与真实执行路径一一对应、可由源码追溯；二：框架原生的提问中断机制改造为创作流程中的人工闸门，让智能体有充分的自主性和长程任务能力的同时，避免大模型与人类的思维出现大幅偏差，模型认为指令有不清、不全的情况时可以向用户提问并澄清；其三，闲置会话回收器（idle reaper）改写为「数据库存档七天＋重开惰性恢复」，用户隔天回到历史会话仍能完整回放；其四，停止按钮转发框架的会话中止端点（`/session/:id/abort`），当前轮次强制停止，不等待模型输出完毕。附录 A 代码块 14–17 分别实现了这4种功能的；代码块18实现了多专家路由与能力断路。

控制面深度嵌合开源 AI 网关 new-api：提供配额唯一信任信息源、消费日志与充值兑换，本项目的独创在于摒弃「阻塞式预扣—回滚」的计费闸门，采用「无状态只读预检＋终态异步投影记账」的嵌合层（见 3.5 表与第七章；控制面各机制的源码节选见附录 A 代码块 2、3、7–13）。

3.4 许可与项目贡献

opencode 为 MIT 许可，new-api 为 AGPL-3.0 许可；new-api 以独立网关服务运行，本项目经其 HTTP API 嵌合、两者是相互独立的著作；配额权威来源与消费日志不算自研成果，本项目的贡献限于上述嵌合层与实现审计；AGPL 组件随发行分发所承担的开源义务，已在项目档案中记录在案。镜头六要素等影音配方吸收自 Higgsfield「Hell Grind」开源规则库（MIT），平台的改写在于将其后端化为自动执行的提示词规范化模块并接入镜头轨迹翻译与场景元数据校验（参考文献逐条可查）。

3.5 工程目标与验证方式

为使各项能力可被逐条检验，关键目标以「目标—机制—验证方式」三栏呈现。表中每一条都可在评审现场复现；本报告的量化证据限于系统产出、工程指标与测试记录；用户可用性对照实验（任务负荷与可用性量表）列为下一阶段工作。

表 3　工程目标与验证方式

工程目标	实现机制	验证方式（现场可复现）

工程目标	实现机制	验证方式（现场可复现）
一句自然语言 → 一张完整创作图	意图编译（沿用编译器「前端→中间表示→校验→后端」的经典分层）：模型只产出 Intent IR，由编译器落图	现场发一句需求，观察画布一次成图（图 4）
非法结构不执行	编译器全图校验：类型不匹配、引用悬空、依赖成环则整份拒绝，不产生半成品图	现场构造一条非法声明，观察整份被拒并给出可执行的错误提示
声明的确定性重放	幂等与往返一致：同一声明反复执行结果一致，图与声明互为可逆	项目内自动化测试锁定，可现场运行测试
人工改动不被覆盖	人工编辑优先（对应版本控制中「人工修改优先于自动格式化」的既有实践）：人工编辑过的对象退出自动布局，但保留排序位置	现场拖动一个节点后触发自动布局，观察该节点不动、连线不乱
审美可锁定、可复用	风格锚定、角色 DNA（即动画工业角色设定表 character sheet 的生成式版本）、身份与关键帧锁定、资产包	现场生成同角色多视角或同主视觉多版位，比对一致性
计费与账目可核验	与开源 AI 网关 new-api 嵌合：提交前做无状态只读配额预检（额度不足即拒单，不产生注定失败的作业），终态由异步投影按确定性定价记账，幂等引用＋状态机认领防止重复扣费；事件账本只追加不修改，上游错误原文入库，展示层仅复现事实、不作评价性改写	源码可现场抽查（`quota-precheck.ts`／`billing-projection.ts`／`job-events.ts`），相关不变量由项目自动化测试锁定（表 9）

3.6 开发过程的自反式迭代

本项目的开发流程与平台的生产流程采用同一套分工结构：模型输出表达、引擎掌握控制流、人保留最终决策（见 4.2）。开发本身就走这套结构：**评测**以真实会话驱动产品执行约定任务，**修复**按「编排者判定、子代理并行实施、验收核对证据」分工，**回归**由自动化测试把关。

迭代按固定链路运行：约定任务、真实会话执行、对过程与结果逐项评分、将问题映射为修复项、并行实施、回归重跑。该链路在项目内部代号为 agent-evolve，不属于本文的对外术语。每轮固定产出评测报告、修复清单与回归证据，按轮次编号归档，任一评分项可回溯到对应的会话记录。

评分如何转化为约束，可以用一个具体案例说明：第九轮评测中，一项长任务的成片视觉品质仅得 2.8 分（满分 10），且过程信息完整而正文未推进；该轮之后，系统在网关侧加入硬截止与看门狗机制，报价与判活端点强制同源，过程信息异常由界面断言覆盖，该类失败在此后的回归与生产观察中未再复发（硬截止、七类看门狗与判活同源的源码节选见附录 A 代码块 19、20）。约束在迭代中单调累积：一类失败一旦被锁定，后续版本不再出现同类问题。7.3 节的 82 次提交是约束的累积，不是同一问题的反复返工。

四、交互范式：人机共笔的可复用工作流

Human–AI Co-Authored Reusable Workflow

本章将本系统的交互方式放入既有方案中定位，并给出对应的系统设计。既有系统大致可分为三类：工作流自动化工具由人预先组装执行流程，机器按图执行（如 n8n、Dify）；节点式生成工具以节点图表达生成过程（如 ComfyUI）；对话式智能体由模型根据自然语言理解进行规划与执行（如 Manus）。

三类方案各有明确优势：确定、可配置，或具备自然语言理解能力。不足也集中于各自的处理对象：前两类以「执行」为对象，流程结构本身不理解意图，设计判断与交付验收留在系统之外；第三类以「对话」为状态容器，理解力随之而来的是易失、线性、不可直接编辑的状态。仅靠其中任一形态，都难以同时满足低表达门槛与可交付确定性。

时代坐标。这个领域的关键词近年快速演进：二〇二四年是「提示词工程」（prompt engineering），二〇二五年是「上下文工程」（context engineering）[4]，二〇二六年被普遍称为「**harness 工程**」（harness engineering，即模型之外的全部工程约束与反馈回路，业界公式写作 Agent = Model + Harness）[5]。创作侧同样如此：二〇二五年的关键词是「氛围编程」（vibe coding）：凭感觉与模型对话，结果不可复现；二〇二六年转向「智能体工程」（agentic engineering）：人不再直接动手，而是编排并监督执行任务的智能体。与此并行，业界对智能体系统的经典二分是「**工作流**」（workflow：流程由预定义代码掌控，控制流归人）与「**智能体**」（agent：下一步由模型自行决定）。本平台的取舍介于两者之间：它既不是人预先写死的工作流，也不是把控制权整体交给模型，而是**由模型编写工作流、由引擎掌握控制流**（the model authors the workflow, the engine owns the control flow）；人的角色也从「监督者」再进一步，成为同一张图上人机共笔的直接编辑方。

由此可以提炼一个研究问题：在影音创作中，如何同时利用模型的理解能力与系统的确定性？本文的回答是把创作状态的位置从易失对话迁移到持久创作图：智能体负责将自然语言意图编译为创作图，人负责在同一张图上进行审美判断与修改，运行引擎按确定性规则执行。

三种既有范式的优劣势对照见下表。后文结构如下：第三章已给出总体架构与工程边界；本章给出创作图的编译与编辑机制；第五章展开该范式下的核心能力；第六章归纳四项核心创新点；第七章给出产物与质量证据；第八章与同类系统对照；第九章总结。

4.1 三种既有范式的优劣势

表 4　三种既有范式与本平台的对照

范式	谁决定流程	状态存在哪里	解决了什么 / 留下了什么
流程编排工具（n8n、Dify 一类工作流自动化平台）	人预先搭好每个节点与连线	人写的流程图与配置	**解决**了确定性与可审计：一次搭成，可稳定重复执行。**留下**三个问题：理解力只封装在节点内部（LLM 节点是预设环节），流程结构本身不认识意图，意图每变一次，人就要重搭一次；产出的是任务数据而非可交付的作品结构；分镜、一致性、质检这些设计判断在流程里没有容身之处。
节点式生成工具（ComfyUI 等开源无限画布类项目）	人手工连节点；二〇二六年起官方智能体接口（MCP／CLI）可让模型替人读写与连接节点	人搭出来的图（以工作流文件持久）	**解决**了「图式执行」：节点生态逾五千包、多模型多媒介皆可入图、模板市场成熟，连图操控也已智能体化。**留下**三个问题：图承载执行拓扑，不承载设计判断（分镜、配乐、节奏、质检仍靠人另行完成）；上游模型的健康、欠费、故障切换与账务不在图内，按算力计费而不按交付验收；产物是素材文件，没有「缺一项即不通过」的交付契约。
对话式智能体（Manus 一类）	模型自己决定下一步调什么	对话上下文（易失）	**解决**了理解力与灵活度：人用自然语言表达，模型自主规划执行。**留下**四个问题：状态寄生在对话里，易失、线性、不可编辑；过程纵有流水记录也是只读黑盒，无法在任意步骤上改写重演；出错后无法局部修正，只能整体重来；成果无法交接给别人继续做。

范式	谁决定流程	状态存在哪里	解决了什么 / 留下了什么
本平台 人机共笔的可执行创作图	智能体声明意图，编译器落图；人随时改写	**图**——持久、可视、可编辑、可回放	智能体提供理解力，编译器提供确定性，人保留审美裁决权与修改权。产物是一张可执行创作图，可回放、可交接、可复用（历史回放的传输瘦身与事件游标源码节选见附录 A 代码块 21）。

三类既有范式的差别可以概括为：流程编排工具由人把定义清晰的流程交给机器执行；节点式生成工具把图的执行交给机器，但设计判断与交付验收仍在流程之外由人完成；对话式智能体由模型自主规划与执行，人没有直接编辑入口；本系统由智能体生成方案并经编译落成创作图，人在同一张图上直接修改，双方共享同一权威状态。这一取舍如何落到工程上，取决于一条关键设计决定，见 4.2。

4.2 Agent Harness 中的约束

本平台有一条关键的设计决定：**把智能体从执行位置移出，仅承担编译阶段的工作**，即智能体只做编译、不做执行。工程约束体现在：模型不允许以逐节点增删的方式直接改写画布，必须先产出**中间表示**，再由**确定性编译器**完成校验与落地。对话式智能体的不可控，正来自它允许模型直接修改作品结构。这与编译器把「源代码」与「机器执行」分离的分层做法一致。

这份中间表示即 **Intent IR**：它不描述执行步骤，只描述**目标结构**——有哪些节点、节点之间如何连接、每个节点承载什么内容与参数。随后交给**确定性创作图编译器**处理：

(1) **全图校验**：检查类型是否匹配、引用是否悬空、依赖是否成环，任何一处不合法就整份拒绝，不会留下半成品图。

(2) **依赖分析与拓扑编排**：自动决定节点分层与执行顺序，人不需要关心先后。

(3) **原子提交**：合法则一次性落图，成功则全成，失败则全不动。

(4) **幂等与往返一致**：同一份声明反复执行结果完全一致；反向把已有的图还原成声明、再编译回去，节点标识保持相等（标识哈希的源码节选见附录 A 代码块 6）。这保证「图」是唯一权威状态，声明只是它的可逆表达。

(5) **可执行的错误提示**：校验失败不返回一句笼统报错，而是给出可执行的修正指引：缺什么就提示什么，名字写错就给出最接近的建议。

这套结构把**概率性的语言模型限制在「表达意图」这一层**，把所有需要确定性的部分交给编译器与执行引擎：既保住大模型的理解力，又让系统达到工程级的可预测性。编译产物同时对外提供接口（可编程调用），这套编译能力本身也是平台对外能力的一部分。这些约束有对应测试逐项背书（清单见第七章表 9）；编译器成环检测的源码节选见附录 A 代码块 1。编译器保证创作图的确定性，但设计质量仍依赖人的判断，由此引出创作图上的人机编辑机制。

4.3 人在环中：可视化编辑与人工编辑优先

创作图落成后，人不只审阅，而是直接在图上继续创作：拖动位置、改写内容、裁剪切分、抠除旋转、增删连线、改变参数。

人机共笔能够成立的关键机制是**人工编辑优先**（Human-Edit Precedence）：人工修改过的节点不再参与自动布局，但仍保留其在排序中的位置，周围节点的连线不会被破坏。它对应版本控制中的既有实践「人工修改优先于自动格式化」，要解决的就是：自动化要整理，人要控制，两者不得互相覆盖。人作出改动后，智能体把每一次改动作为既定前提继续推进，而不是覆盖。

配套的交互设计对应三项既有实践：**确认闸门**（Confirmation Gate；human-in-the-loop approval）：产生费用的操作先经用户批准，用户保留否决权；**批量锚定**（Batch Anchoring）：将首个满意结果实际使用的参数冻结为基线（frozen baseline），批量任务据此复现，避免整批漂移；**渐进式披露**（Progressive Disclosure）：常用参数前置、进阶参数收起，取值一律来自真实调用记录。两个方向的修改作用于同一张图，最终决策与交付归人。

4.4 创作状态的迁移：从对话到图

上述差异，归结起来是一句话：**创作状态的位置变了**。

在对话式范式里，作品的真实状态散落在对话上下文中：线性、易失、无法被直接编辑；删掉一条消息就可能失去一段构造；会话一长，当前做到哪一步也难以说清。在本平台的范式里，真实状态是一张**持久的图**：

表 5　状态迁移带来的性质变化

性质	对话即状态	图即状态
可否直接编辑	只能靠再对话一轮，间接重试	可以直接改任何一处，改动立刻生效
可否复现	不可复现，模型每次都可能不同	声明幂等、往返一致，可原样重放
可否审计	不可审计，过程混在对话内容里	每一步都是可见的节点与连线
可否交接	无法交接，他人无法接手对话上下文	整张图可交接，别人接着改
跨媒介	各模态独立对话，状态互不相通	五种媒介在同一张图上成为上下游工序

迁移之后，创作状态成为持久且结构化的对象：可保存、可检查、可复制，可交给他人继续，也可被第三方程序调用。设计能力由此从个人经验变成平台资产。

4.5 工作流的构成：一套类型系统承载生成、资产与编排

这张图不是自由画布，而是一套有类型、有语义的图：图上共有十二类节点，覆盖三类角色：

(1) **生成类节点**：图像、视频、音频、三维场景、三维导演台——承载真实的生成与渲染工序。

(2) **资产类节点**：带角色的参考图、角色卡、场景卡、提示词备忘、配置——承载审美标准与可复用资产。

(3) **组织类节点**：文本与成组——承载说明、结构与时序关系。

关键在于**连接即语义**（Connection as Semantics）：图中的边是有类型、有语义的，连线即声明数据依赖与传递，与数据流编程（dataflow programming）一致。提示词节点连到生成节点，文本自动作为提示前缀注入；带角色的参考图节点连上去，图像与角色定义一并传递；三维场景渲染出的构图可作为视频生成的参考。这些规则由平台统一解释，人工操作与智能体操作走同一条语义通道，不存在「人能做的」与「智能体能做的」两套逻辑。

五、核心能力

5.1 来自自然语言需求的专业资产生成

平台的核心能力是：用户以自然语言提出需求，AI 交付可直接进入生产的专业资产。创作过程由七类专业角色分工完成，每个角色的产出都是结构化的设计决策，而不是候选素材。

表 6　七类专业角色交付的设计产出

角色	做的是什么设计	交付产出（交付规则强制齐全）
编剧	故事与结构设计	故事节拍、镜头表
视频导演	镜头设计	镜头清单、成片时长、模型建议
音乐导演	听觉与情绪设计	情绪需求、歌词、成曲音频、时长来源
剪辑师	时间与节奏设计	剪辑时间线操作、成片时长
三维导演	空间与运镜设计	转台动画、静帧、工程文件、可交换模型
动效导演	动态与转场设计	动效规格
网页三维导演	交互空间设计	场景描述与资源清单

以「做一支 60 秒品牌短片」为例：用户只需描述产品、受众与想要的感觉。具体地，编剧给出故事节拍；导演把节拍拆成镜头清单并逐镜生成画面；剪辑师把镜头接成时间线并控制总长；音乐导演根据情绪需求作曲并混音；最后成片带着音轨交付。这条流程中，用户没有填写参数表。

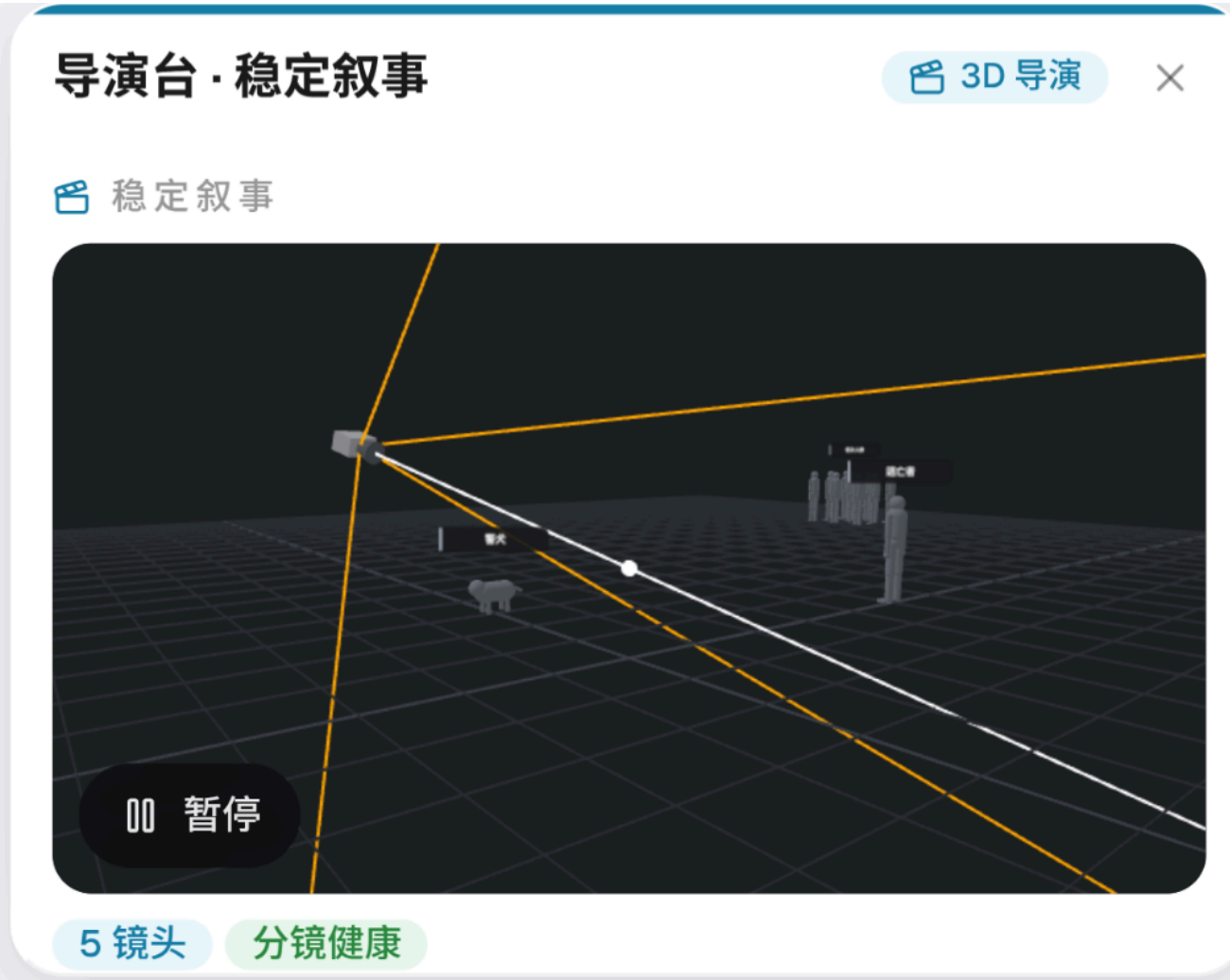


图 2　AI 自主完成的三维设计：三维导演自行编排的场景——奔逃者、警犬与人群的位置、机位与运动路径全部由 AI 设计，分镜状态由系统自动校验为「5 镜头 · 分镜健康」

5.2 审美一致性：资产化的锁定方法

在交付层，一致性问题最为突出：它既是 AI 创作的主要失败点，也是素材交付与设计交付的分界。本平台的做法，是把一致性固化为一套可复用的资产体系：

表 7　一致性资产体系

机制	解决什么问题
风格锚定	一组作品在换尺寸、换版位、换媒介时，仍锁定同一套调性而不漂移
角色 DNA 转面	同一角色产出多视角一致的转面图，解决「第二张就换脸」
关键帧锁定	把满意的画面定为基准，后续镜头围绕它保持一致
角色与形象锁定	同一个人物、同一个形象跨多次生成保持身份一致
资产包	把一套视觉资产（风格、角色、参考图、配方）打包，跨生成、跨项目反复复用
一稿多版位（即广告美术的多尺寸改版）	同一主视觉批量改写为多个尺寸与版位，主体、构图关系与风格保持不变

这样，系列化生产不必逐张重做：第一件作品定稿后，整套物料与整套片子沿用同一审美标准，无需逐张重做。一致性资产解决「产出如何稳定」；输入侧还需要解决「需求如何表达」。

5.3 提示词规范化：从口语表达到镜头语言

需求的表达问题集中在提示词上。多数用户无法产出合格的提示词，原因是专业影像描述有一套独立的语法。本系统在后台运行**提示词规范化模块**，自动承担这套语法：

(1) **镜头六要素补全**。系统按「机位＋主体＋动作＋环境＋风格＋光照」六要素检查用户的描述，缺哪个要素就补哪个要素；用户原话里的部分一律保留、不被改写。

(2) **情绪具象化**。不写「她很害怕」，而写「下颌收紧、鼻翼翕动、呼吸变浅、肩膀内扣」，即把情绪标签翻译为具体的肌肉动作描述，该改写方式对应表演指导中常用的可见动作写法。

(3) **剔除空泛形容词**。把「很美」「高级感」这类没有视觉指向的词，替换为具体的光线、构图与材质描述。

(4) **负向表达正向化、歧义表达消歧**。把「不要出现××」改写成正向声明，避免模型理解偏差。

(5) **一稿起手**。平台内置风格库 106 条、场景模板 14 条、镜头要素配方模板 10 套，用户可以从成熟配方起步，而不是从空白开始。

这些处理都在后台完成，界面回显的仍是原始表述。因此，会不会写提示词不再是使用平台的前提；六槽拆解与缺省归类的源码节选见附录 A 代码块 5。

5.4 设计质量控制：确定性门禁与交付规格

生成完成不等于达到质量要求。平台内置 17 道确定性门禁用于分镜质检，逐项判定镜头表在结构、连贯与可执行性上的问题。平台同时以交付规则约束每个专业角色的交付物：**缺任何一栏，组装闸门即不通过**；需要质检的角色还须提交内容签名，篡改任一字段即校验失败。17 道门禁的编号清单与阻塞判定源码见附录 A 代码块 4，执行侧交付诚实度判定见代码块 22。

据此，本平台对「完成」的判定以交付物为准：交得出齐全且可验证的成果。设计质量靠它稳定，AI 产出靠它进入生产交付。门禁与交付规则保障单个产出的质量；跨媒介的协同生产见 5.5。

5.5 跨媒介协同：五类媒介统一生产与一稿多版位

跨媒介协同要求这些产出能够连接为同一部作品。图像、视频、音乐、语音与三维因此是同一张创作图上的不同工序，而不是五个互不相通的工具：三维场景渲染出的构图可以作为视频生成的参考；视频成片可以直接进入剪辑与配乐；音乐的情绪需求为整支片子统一听觉基调。

同一主视觉还可批量改写为多个尺寸与版位（内置 13 个常用版位，覆盖方图、竖图、画册与通栏），主体与风格保持一致，对应「一张图要发十个渠道」的多渠道适配需求。

5.6 资产沉淀：生成结果的自动归档与复用

平台的产出以资产形态自动沉淀。生成工作台把需求输入、参数选择与结果呈现集中在同一界面；每次决策的参数与费用在界面可见；生成结果自动归档入生成历史，按时间分组并支持检索与筛选。归档动作由系统完成，无需用户手动整理。

历史卡片上最常用的两种后续动作都是复用：「再来 N 组」按原配方原样重跑，参数不改动，提交前先给出批量报价；「存为卡片」把结果锁定为角色卡、场景卡或风格卡，挂到画布上，此后所有生成自动携带这张图。该机制与 5.2 所述一致性资产体系共用同一套资产表示。

跑批闭环：一次执行同时产出成品、记录与经验

单张生成之外，平台还承接以创作图为单位的**批量跑批**（一条工作流一次跑完整条生产线）。这类任务的执行状态在运行期只存在于执行引擎的内存中，跑完即散——若不主动落盘，历史页看不到任何「一次工作流跑完」的痕迹，用户也无从复用一条已经验证过的生产配置。平台的解法是在跑批收尾时开一条**单向闭环**，让同一次执行沿着一条因果链依次沉淀出三样东西：

(1) **成品索引**——把这次跑批的全部产出一一登记（含逐档内容指纹），使「宣称交付」与「实际落盘」可逐项对账，而不是凭结束状态自述。

(2) **可查的记录**——把执行图投影成一条历史记录：节点、连线、逐节点实付费用与失败节点一并入档。历史卡片上直接渲染这张结构缩图，用户看到的不是一张缩略图，而是这次生产线的真实形状：哪一环失败、哪一环最贵，一眼可辨。

(3) **可召回的配置**——把这次跑批的配置抽取为一条记忆入库，此后可跨会话被召回，作为下一次同类任务的起手配置。

这条链上有一处设计值得单独说明，因为它直接决定了沉淀物的可信度：**记忆的内容由机器抽取，不经过语言模型**。入库的正文就是那条跑批配置本身的序列化结果——位元可复现、跨环境一致、零幻觉。语言模型在此只承担一个**闸门**角色：判定「这条跑批值不值得进库」（判据见第六章创新点五）。如此分工的理由是：让模型去「总结」一条已经结构化存在的配置，只会引入失真，而配置的复用价值恰在于它是精确的。

同一纪律也贯穿抽取的边界处理。记忆条目有硬长度上限，超长时平台的让步顺序是预先定义好的：优先保留配置本体，其次按剩余预算逐条裁剪节点标签，最后才整体截断并在尾端显式标注省略——**从不抛错，也从不谎报全量**。若抽取失败即抛错，这条记忆会静默消失且无人察觉；若截断而不标注，用户会把残缺配置当作完整配置复用。两者都是「看起来成功」的失真，因此都被排除。

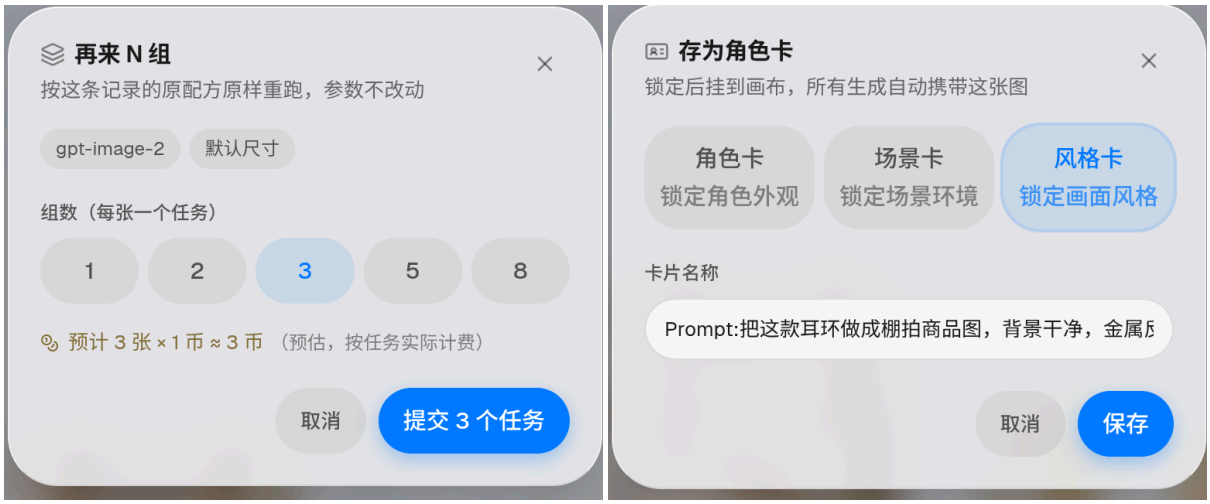


图 3　两种后续动作：左为按原配方重跑并预先报价；右为锁定为角色卡／场景卡／风格卡并挂上画布。

六、核心创新点

创新点一　人机共笔的可执行创作图　创作系统首先要回答的问题是：作品的真实状态存放在哪里，人与机器能否分别编辑它。既有工作流工具由人预先定义流程结构，流程本身不理解意图；对话式智能体将状态寄存于对话上下文，状态易失、不可直接编辑。本文的处理方式如 4.2 所述：智能体不直接执行生成，模型产出 Intent IR，经确定性编译器校验后原子落成创作图；人在同一张图上直接编辑，人工编辑优先。创作状态由此迁出易失对话、落在持久图上（性质分析见 4.4），整张图可审计、可回放、可交接，也可程序化调用。

这一优先关系的具体形式是：人工改动过的对象在后续自动编排中保持原位，不被移动或覆盖。

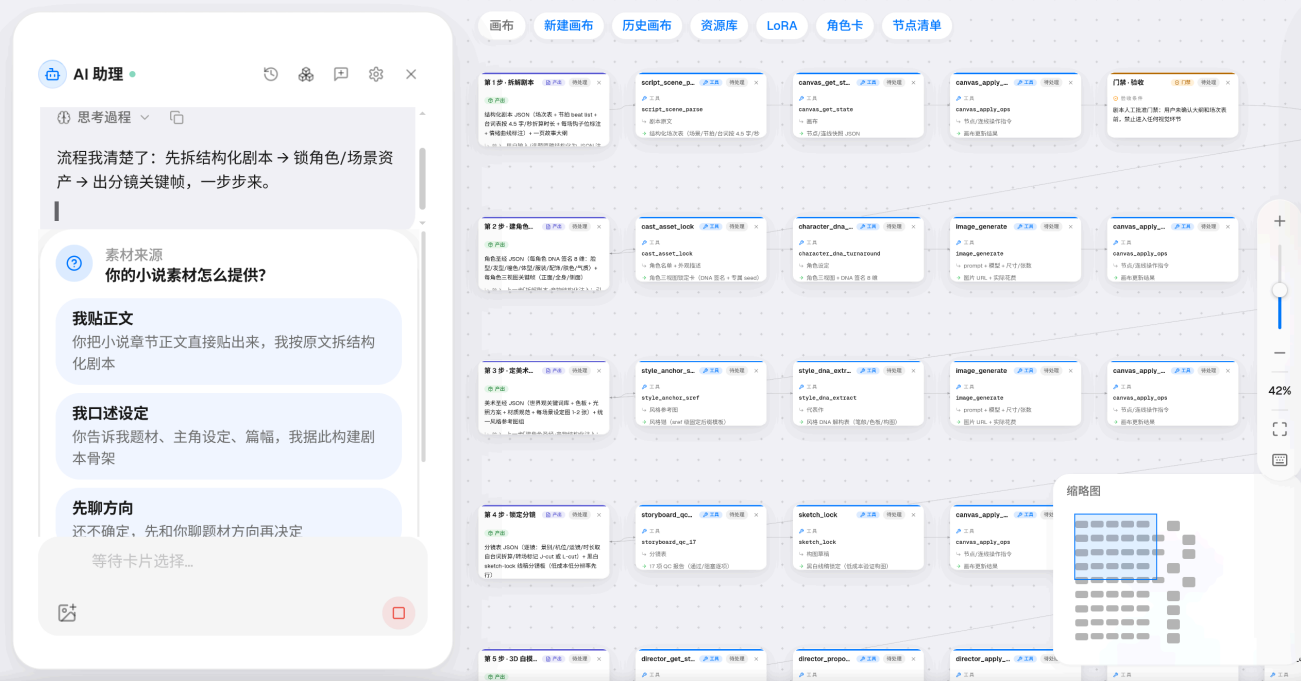


图 4　平台真实运行截图（以套用「小说转影音」模板为例）：一句自然语言需求发出后，整张创作图一次性落成而非逐节点堆叠；智能体停在人工决策点等待选择。

创新点二　设计决策的规则化交付　第二个问题是：「生成完成」并不等于「生产完成」，产出齐备由谁保证。现有工具将 AI 定位为素材生成器，输出候选内容后由用户完成判断、组织与修补，产出是否齐备缺乏机制保证。本文将专业角色与交付规则引入生成流程：七类专业角色各自受交付规则约束，交付物缺项则组装闸门不通过；需要质检的角色须提交内容签名，字段篡改即校验失败。进一步地，AI 的产出从候选素材转变为可验证的专业资产（实现细节与流水线示例见 5.1，验证记录见图 5、图 6）。



图 5　平台真实运行截图：一部短片的自主生产过程。左为创作骨架定案，中为编剧与音乐导演双线并行派遣，右为分镜稿定稿。



图 6　平台真实运行截图：番剧世界观自主设计。左为需求与派遣规划，中为四位主角设定，右为 12 话叙事结构。

创新点三　审美一致性的资产化　第三个问题是：跨多次生成乃至跨媒介，一致性如何维持、如何转移。规则化交付解决了产出齐备，不保证产出之间一致。生成模型的漂移发生在采样内部：固定 seed、LoRA、参考图等单点手段能锁住单张图，锁不住整个系列：同一角色跨镜头身份漂移，同一系列更换尺寸后调性变化，跨媒介一致性更难维持。本文把风格、角色、关键帧与参考资产固化为可版本化、可复用、可交接的资产：具体机制包括风格锚定（跨尺寸锁调性）、角色 DNA 转面（多视角一致）、关键帧锁定、角色与形象锁定、资产包与一稿多版位（同一主视觉批量改写为 13 个常用尺寸而主体与风格不变，机制明细见 5.2）。这样，审美一致性不再依赖重复生成与人工筛选，而成为可管理的资产。

创新点四　提示词规范化的自动改写　第四个问题是：说什么与如何说得让系统执行，是两种不同的能力，专业语法不应成为使用系统的前置条件。多数使用者难以直接写出符合专业影像语法的提示词，这套语法（镜头六要素公式、情绪的可描述方式、正向表述与消歧规则）通常要用户自己学习。本文将这一层移入系统：提示词规范化模块完成镜头六要素补全、情绪具象化（将「害怕」改写为下颌收紧、鼻翼翕动、呼吸变浅等可见的肌肉动作）、空泛形容词替换、负向表达正向化与歧义消解，同时保留用户原始表述。用户无需先掌握这套语法，即可获得符合镜头语言规范的生成描述（模块设计见 5.3）。

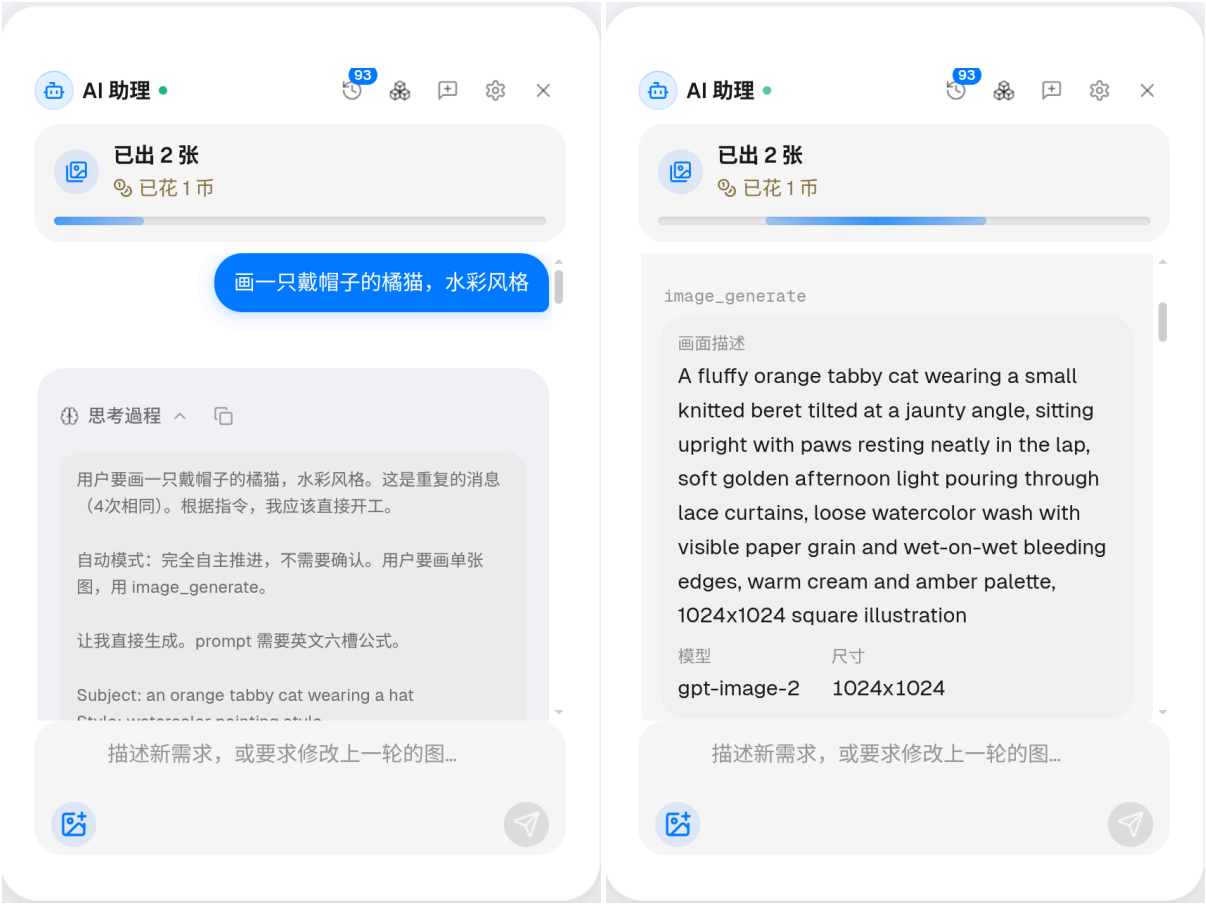


图 7　平台真实运行截图：左为用户输入「画一只戴帽子的橘猫，水彩风格」及智能体的需求要素拆解；右为系统实际提交的提示词（补齐姿态、光线、材质、色板与尺寸）。

创新点五　Agent Harness：把不确定的判定从语言模型手里接管过来

第五个问题是：哪些事不该交给语言模型做。前述四项贡献都建立在「智能体负责理解、引擎负责确定性」这一分工之上，但分工的边界画在哪里，本身是一个可继续推进的问题。以判断题为例——本轮的用户输入是否夹带指令注入、这次任务该派给哪位专家、这段交付是实质内容还是空壳、这条配置值不值得沉淀——传统做法是把判据写进提示词，请模型「顺便判一下」。这条路有三个固有问题：判定与生成共用一个概率分布，结果随之波动；判定过程本身要消耗生成长度与时间；而最要紧的是，判定失败时没有任何机制能把它与「模型今天状态不好」区分开来。

本平台的处理方式是把这类判定整层移出语言模型，交给一个**非自回归的决策模型**（业界以 System One 范式著称的代表实现为 Jev；平台另以同范式的开源模型 Laya 作同等契约的兜底）。它与语言模型的关键差别不在参数量，而在输出形式：它不生成文本，只接收结构化的 state 与一组**带类型的判断题**，直接回传数值与标签。判定因此从「生成」变成「测量」——可标定、可复现、可审计。这是平台的方法论在 Harness 层的一次延伸：不只是让引擎接管执行，而是进一步接管**判断**。

Harness 的价值，体现在它如何把这个能力用到极致。以下四点均为平台侧的工程设计，与所选判定模型无关：

(1)　**批量提问，一次往返**。System One 范式允许在单次调用中并行提出多道互不干扰的判断题，加题不增加往返延迟。平台据此把一处决策点的全部判定合并成一次调用：收尾时一次问完「简报是否如实」与「每一路交付是否空壳」，派遣时一次取回「任务性质分类」与「该用哪个生产模板」。判定层因此不是链路上一串串行的关卡，而是一个几乎透明的旁路。

(2)　**便宜的前置过滤**。并非每个输入都值得一次判定调用。平台为注入扫描配了一道零成本的字符级预筛：不含任何可疑模式的输入直接放行，判定调用只花在少数真正可疑的样本上。多数路径因此完全没有判定延迟——这与「先花大钱再后悔」的典型 AI 用法正好相反。

(3)　**判定层的失效绝不断链**。一个新增的判定依赖会成为新的单点。平台的处理是：判定层全部采用 **fail-open**（失败即放行，退回原有行为），判定结果只服务于增强而非阻断，且真正的安全硬门禁设在另一条与判定模型无关的确定性链路上。判定层整体失效时，平台的行为**逐位等价于它上线之前**——这是可验证的退化保证，而不是「尽力而为」。

(4)　**可归因的兜底**。判定模型按量计费，额度耗尽会导致所有依赖它的判定同时断供。平台以同等契约的替换实现承接（见下），并在日志中显式标记本次判定来自哪一方，避免「静默换供应商」让判据来源事后无法追溯。

这套 Harness 的可靠性不靠叙述，而靠标定。以注入扫描为例：平台构造了覆盖中英文的越狱样本（覆盖既有指令、绕过计费、假冒管理员、角色扮演、把注入藏在研究结论里）与正常创作样本各一组，实测攻击样本的判定值落在 0.98–0.99，而正常创作样本最高仅 0.10，阈值取在两者之间留双向余量。这种干净的两端分离，正是「判定被当作测量来做」的直接后果：分离度可以被测出来，因而可以被信任；而一个概率性语言模型的「顺便一判」无法给出这样的曲线。同一套标定也用于其余判定点（专家路由、模板选择、记忆入库闸）。平台在推进节奏上遵循同一纪律：先以只记录不干预的方式观察真实分布，达到样本量后再转为权威判定，而非凭直觉设阈值。据此，注入扫描、专家路由、模板判定与记忆入库闸已转为权威判定（越狱样本实测被拒、正常创作照常放行、路由与模板注入在运行日志中逐次留痕）；而诚实性判定因其判据涉及对交付内容的定性评价、且样本量仍在积累，目前仍处于只记录不干预的观察阶段——**平台不把「已观察」当作「已生效」**。

需要明确的是平台的贡献边界：判定模型本身是外部已有能力，平台既不训练也不修改它。平台的工作在于**把它放进一条正确的因果链里**——决定哪些判定该移出语言模型、如何批量提问、如何前置过滤、失效时如何退化、兜底如何同构、阈值如何标定。这些是 Harness 层的工程，也是本平台认为自身最有价值的部分：模型能力可以被任何人调用，但把模型能力组织成一条稳定、可审计、可退化的生产线，是平台真正的产出。

补充证据：可现场复现的关键机制



图 8　平台真实运行截图：三维导演台。用户输入一句不含任何参数的需求，智能体自行规划人偶、动物、人群与多机位调度（场景画面见图 2）。

五个问题依次对应五项贡献：创作状态存放在哪里、产出齐备由谁保证、一致性如何维持、需求如何表达，以及**哪些判定不该由语言模型做出**。前四者共同构成平台的方法论层，与第三章的三层架构、第五章的核心能力一一对应；第五者则作用于 Harness 层自身——它不新增产品能力，而是让前述四项贡献的判定环节变得可标定、可复现、可退化。平台认为后者与前者同等重要：一项能力能否被信任，取决于它的失败边界是否清晰，而非它的宣称是否动人。

新颖性说明。在本文检索范围内，尚未发现同时具备以下两项特征的同类公开系统：其一，智能体不直接修改创作图，仅生成声明式中间表示，由确定性编译器完成校验与落地；其二，人工改动在后续自动布局中非覆盖式保留，且维持排序位置（与同类工具的逐项对照见第八章）。

七、成果验证

7.1 产出验证

下表所列产出均非人工拼装，而是用户提供输入材料与一句自然语言需求后，由生产流水线独立执行的结果：系统自行拆解剧本、锁定角色 DNA、确定美术设计方案、生成分镜并通过 17 道门禁校验、补充关键帧、配乐、组装剪辑时间线。本节产出记录均来自平台运行日志与对应媒体文件，规格与时长由媒体探测工具（ffprobe）实测，测量命令与原始文件留档于项目档案：

表 8　实测产出清单（每一件对应的自动工序）

产出	规格（实测）	用户输入 → 系统独立完成的工序
完整短片《大明最后的十七年》	1280×720 / 64.48 秒 / H.264 + AAC	贴一段题材 → 拆结构化剧本、建角色设定、锁分镜、渲染关键帧、合成镜头、组装带音轨成片
多镜头拼接片段	1344×768 / 36.26 秒 / 含音轨	一句题材 → 片头＋六个镜头自动分镜、逐镜生成、按时间线拼接

产出	规格（实测）	用户输入 → 系统独立完成的工序
三维转台动画	1280×720 / 24fps / 5.0 秒	一张产品图 → 三维导演定机位与环绕运动、无界面渲染器实渲
短剧海报与视觉素材	1920×1080 / 10–12 秒	一句剧名与调性 → 主视觉＋多版位批量改写，同一条流水线的旁支产出
原创配乐	65.7 秒 / M4A	一段画面情绪 → 音乐导演出情绪需求、作曲、混音成完整音轨



图 9 《大明最后的十七年》关键帧：自成片第 4、14、24、34、44、58 秒抽取，画面为火海宫阙、长廊、殿前、烈焰、火把阵列与雪夜；全部 64.5 秒出自同一张创作图。

7.2 质量基线：不变量与约束锁定

本文用具名测试断言逐项锁定开发中出现过的五类失败模式。表 9 列出五类失败模式及其对应约束与锁定证据；五类模式均真实发生过，在当前回归测试集下未再复发。该做法遵循 harness 工程中的一条实践：把已观察到的失败转化为可复验的环境约束 [5]。

表 9　被约束锁定的失败模式（当前回归集内）

失败模式	无此约束时发生什么	约束它的机制	锁定证据
半成品图落画布	编译半途失败留下残图，用户面对一张既不可执行也不可回滚的坏图	全图校验：类型不匹配、悬空引用、依赖成环，任一不合法**整份拒绝**；合法才原子提交	`layout.test.ts`，断言「图与声明往返一致：坐标不变、units 清空、悬空连线被丢弃」——图→声明→图往返恒等，悬空引用必被拒
自动排版覆盖手动摆放	人刚拖好的节点被机器一键整理抹掉，双方编辑互相覆盖	人工编辑优先：人工编辑过的对象退出自动布局，显式意图优先于自动接线，排序位置保留	`semantic-nodes.test.ts`，断言「手动在前（显式意图优先）」——手动参考不被自动接线覆盖；现场可复验（3.5 表：拖节点后触发自动布局，该节点不动）
重复扣费／重复退款	网络抖动触发重试风暴，同一笔生成向用户重复扣费，或退款重发导致超额入账	幂等引用＋状态机认领：终态迁移只发生一次；结果含糊的退款先冻结、不重发	`billing-ledger.test.ts`，断言「全额退款只触发一次终态迁移，且不重复产生外部入账」「含糊退款先冻结，后续重试不产生第二次入账」

失败模式	无此约束时发生什么	约束它的机制	锁定证据
账目失真	上游真实消耗了算力而平台不记账（未计费消费），或上游未消耗却向用户扣费（错误扣费），两个方向的账目失真	核销判定以上游**尝试证据**为准：被客户端拒绝的 4xx 状态码（请求本身不合法）不记账；发生过 5xx（上游故障）或 2xx（已出账）尝试的必记账	`billing-projection.test.ts`，断言「无上游尝试的失败作业不核销、不写入任何账目」；504 超时尝试则断言「按真实上游消耗写入核销记录」
额度不足仍接收作业	注定失败的作业占用队列与上游配额，用户空等一轮，平台无谓抬高失败率	无状态只读预检：提交前读取网关侧配额权威源，不足即 402 拒单；网关不可达时回落原扣费路径，保证任务可提交	`quota-precheck.test.ts`，断言「门禁开启且额度不足时按既有 402 契约拒单」；网关不可达时断言「原样回落既有预扣路径」

上述五类模式与分镜门禁、组装闸门、幂等重放等其余约束（具名测试断言本体与引擎侧幂等守卫、逐镜展开的源码节选见附录 A 代码块 23–25），由 1,816 个自动化测试用例（202 个测试文件）中的**具名断言**全部覆盖：每类失败对应一条断言，任何改动破坏其中一条，对应测试即告失败。上述结论仅适用于本文验证的场景与当前回归测试集。

7.3 开发过程与迭代记录

(1) **版本演进：**自 2026 年 6 月 5 日首次对外部署至同年 9 月 12 日，持续迭代三个多月；现行生产代码仓库自 8 月 14 日起 82 次提交，将图像、视频、音乐、语音、三维五类能力接入同一张创作图。

(2) **迭代方式：**迭代以「评测—修复—回归」为主线：每轮以真实会话执行约定任务、逐项记录问题、修复后重跑；该开发机制的说明见 3.6。

(3) **评测记录：**本文涉及的评测报告、修复映射与回归记录均留档于项目档案，本报告《证据索引》即由其整理，可现场抽查任意一条。

表 10　开发阶段划分与验证方式

阶段	时间	主要工作	验证方式
原型搭建	2026-06	平台成形与首次对外部署	部署记录与文件清单（项目档案）
能力扩展	2026-08-14 起	五类生成能力接入同一张创作图	82 次提交与全量回归测试
评测循环	2026-08 至 2026-09	十轮「评测—修复—回归」	评测报告、修复清单与表 9 断言

平台已上线运行（www.eimu.dev），带有用户体系、支付通道、状态监控与开放接口文档。作为在线系统，其可靠性机制包括三项。第一，任务领取采用数据库行级锁的跳过等待语义（`FOR UPDATE SKIP LOCKED`），多工作进程并发互不冲突，在飞任务数与供应商、账号、能力三层并发上限取最小值。第二，计费与开源 AI 网关 new-api 嵌合（见 3.3），阻塞式预扣改为只读预检加异步投影记账；僵死的计费作业按年龄门槛自动回收并全额退款，网关不可达时自动回落降级路径，对外交付规格不变。第三，事件账本采用 append-only 结构，配合全链路探针端点，单次生成故障的排查可沿单条时间线完成；另有七类看门狗机制避免会话长时间无响应。系统表现出的行为一致性（不重复扣费、无静默失败），由上述不变量与测试共同保障。

八、与同类方案的对比

本节以二〇二六年九月同类工具的公开现状为准，从设计与交付的角度对照，不以功能数量为指标。对比的核心差异在于**交付边界**：同类工具解决的是执行层问题：图能否运行、任务能否完成；本平台解决的是交付层问题：产出是否达到可交付标准。

表 11　能力对比：典型同类产品 vs. 本平台（各产品能力描述以其官方站点与代码仓库为据，见参考文献）

维度	典型同类产品（n8n／ComfyUI／Manus／Adobe Firefly 等）	本平台
产出物	单张图像或单段视频（素材）	分镜表、镜头清单、成片、配乐与三维场景，经交付规则与组装闸门通过后组装
设计由谁做	人做设计，模型做生成；智能体接口可代劳连图跑图，不代劳设计判断	AI 承担设计决策，人负责需求提出与最终决策
协作范式	生成为一次性交易：模型输出候选，人工筛选	同一张创作图上的人机共笔：智能体编译落图，人直接编辑，人工编辑优先
创作状态在哪	对话记录随会话消散；工作流文件虽可持久，存的是执行配置，不存设计语义	持久化的图：可编辑、可回放、可审计、可交接
跨媒介	媒介靠节点包逐个扩展，跨媒介拼接靠人手工连边与手工对齐	图像／视频／音乐／语音／三维同一张图上的上下游工序
一致性	有固定 seed、LoRA、参考图等单点手段，能锁单张、锁不住系列：跨镜头一致性靠人逐图维护	风格锚定跨尺寸锁调性；角色 DNA 多视角一致；关键帧与身份锁定；资产包跨项目复用
提示词门槛	提示词由用户撰写，质量取决于个人经验	提示词规范化模块在后台补全与改写
质量控制	无原生交付质检（社区节点可评估单图），成品与否由人工判定	17 道确定性分镜门禁＋交付规则，缺一项即不通过
可编程性	多数产品封闭；开源者已提供运行图的 API／CLI／MCP，但接口面向执行调度，不含报价、验收与账务语义	创作图编译器对外开放接口，可被第三方程序调用
上手方式	模板市场丰富，但模板是执行配方，不承载设计方法论与镜头语言	内置 106 条风格库、14 条场景模板、10 套镜头要素配方，并提供可一键复刻的公开案例

对照取其所长：节点式工具在图的执行、模型生态（逾五千节点包）、模板市场与智能体操控接口（MCP／CLI，官方内置）上处于领先地位，本平台不在执行端与其竞争。

九、总结与展望

本文从生成能力过剩与设计能力稀缺这一矛盾出发，设计并实现了自动化影音设计平台 Eimu。在方法论层面，本文提出人机共笔的可执行创作图：智能体仅编译不执行（机制见 4.2），人在同一张图上直接编辑，两种编辑共享同一状态，并保证人工编辑优先。由此，创作状态从易失对话迁移为持久图结构，创作成为可工程化管理的过程（机制与性质见第四章）。

在系统层面，平台实现了七类专业角色的设计能力层、审美一致性资产体系、提示词规范化模块、17 道确定性分镜门禁，以及覆盖五种媒介的 56 个生成模型与开放接口。系统已部署上线（www.eimu.dev），

实测产物包括 64.5 秒完整短片、36.3 秒多镜头拼接成片、65.7 秒原创配乐与三维转台动画，表 8 所列各项均有原始文件可供核对。

本文的出发点是一个判断：生成能力已趋于普及，设计能力仍然稀缺。平台的各项机制——意图编译、人机共笔、一致性资产与规则化交付——均围绕这一判断展开。如第一章所述，生成模型会持续换代，而设计判断、审美一致性与协作方式不会随之自动获得，这正是本文所处理的问题域。

下一步，本文将开展人机分工效率的使用者研究，提升设计判断的细粒度与可解释性，推动审美资产跨用户流动，并把创作图发展为可被第三方接入与嵌入的开放基座。本文的边界包括两点：其一，人机分工效率的对照使用者研究尚未完成，已列为下一阶段首要工作；其二，分镜质检为确定性规则判定，不等价于人类审美评价。

附　录

附录 A　关键机制源码节选

正文的关键机制断言都可以沿下列二十五段源码追溯到实现。块 1–13 与块 23 出自应用仓库，评审现场可按路径与行号核对；块 14–22、24–25 出自智能体引擎 agent-gateway，部署于作品服务器 /opt/agent-harness/，随在线演示环境核对。源码注释与测试文案按原样保留，不作翻译。

代码块 1｜确定性编译器：成环检测与整份拒绝　对应 4.2「依赖成环则整份拒绝，不留下半成品图」。
来源：`src/lib/workflow-dsl/compile.ts` L194–199、L239–247 逐字摘录：拓扑排序未收敛（存在环）即放弃编译，由环检测生成可执行提示。

```
  }
  if (topoOrder.length < steps.length) {
    const leftover = steps.map((s) => s.id).filter((id) => !satisfied.has(id));
    errors.push(...cycleErrors(leftover, needsMap));
    return { errors, topoOrder: null };
  }
// ……（此处摘录略去连续行）
    if (remaining.size === 0) break;
    const cycle = findCyclePath([...remaining], needsMap);
    if (cycle.length === 0) break; // 防禦: 不可達
    errors.push({
      step: cycle[0],
      field: "needs",
      message: `依赖链 ${cycle.join(" → ")} 形成循环依赖；把其中一方的 needs 移除即可成链`,
      hint: "移除环中一条 needs 依赖",
    });
```

代码块 2｜无状态配额预检：映射缺失与额度不足均拒单　对应 3.3「额度不足即拒单，不产生注定失败的作业」与 fail-closed 承诺。
来源：`src/lib/generation-core/quota-precheck.ts` L139–147、L151–159 逐字摘录：映射缺失 fail-closed 拒入，额度不足按 402 契约拒单。

```
  const tokenId = await findNewApiTokenIdForBizKey(input.bizKeyId, deps.sql);
  if (tokenId === null) {
    console.error(
      `[quota-precheck] fail-closed: no new-api token mapping for biz key ${input.bizKeyId}; rejecting
submission per migration 012 contract`,
    );
    return { ...CONTROL_PLANE_UNAVAILABLE };
  }

  const precheck = await checkQuotaAvailable(
// ……（此处摘录略去连续行）

  if (precheck.verdict === "insufficient") {
    return {
      ok: false,
      status: 402,
      type: "insufficient_quota",
      code: "insufficient_quota",
      message: INSUFFICIENT_QUOTA_MESSAGE,
    };
```

代码块 3｜终态投影记账：唯一幂等键认领，防重复扣费　对应 3.3「幂等引用＋状态机认领防止重复扣费」。
来源：`src/lib/generation-core/billing-projection.ts` L117–128 逐字摘录：以唯一幂等键 INSERT ... ON CONFLICT DO NOTHING 认领记账权，认领失败即跳过。

```
        summary.skippedUnknownPrice += 1;
        console.error(`[billing-projection] skip job ${jobId}: cannot determine base price
(kind=${row.kind} model=${row.model_id})`);
        continue;
      }

      const claimed = await sql<Row[]>`
        INSERT INTO billing_operation (id, job_id, operation, sequence, amount, status, idempotency_ref,
executor_id, attempt_count)
        VALUES (${randomUUID()}, ${jobId}, 'commit', 1, ${amount}, 'pending',
${`projection:${jobId}:commit:1`}, ${input.executorId}, 1)
        ON CONFLICT (idempotency_ref) DO NOTHING
        RETURNING id
      `;
      if (!claimed[0]) continue; // 已投影或另一 worker 併發認領
```

代码块 4｜17 道分镜门禁清单与「任一未过即阻塞」判定　对应 5.4「缺任何一栏，组装闸门即不通过」。来源：`src/lib/shortdrama-domain/index.ts` L168–187、L404–405 逐字摘录：17 道门禁编号成表，任一项未过则整单状态为 blocked。

```
const STORYBOARD_GATE_IDS = [
  "shot_identity",
  "shot_order",
  "scene_binding",
  "duration_bounds",
  "segment_duration",
  "prompt_present",
  "characters_bound",
  "scenes_bound",
  "character_lock",
  "scene_lock",
  "keyframe_present",
  "last_frame_present",
  "audio_anchor_present",
  "aspect_ratio",
  "dialogue_alignment",
  "continuity",
  "export_readiness",
] as const;

// ……（此处摘录略去连续行）
  ];
  const status = gates.every((item) => item.passed) ? "succeeded" : "blocked";
```

代码块 5｜提示词规范化：六槽拆解与缺省归类　对应 5.3「按六要素检查描述、缺哪个补哪个，用户原话保留」。来源：`src/lib/invisible-prompt-director.ts` L318–334 逐字摘录：纯函数按句读切分、正则归类六槽，归不进槽的原话归类进主体槽。

```
/**
 * 從（優化後的）prompt 提取六槽結構化視圖。免費層展示用——把長 prompt
 * 拆成六個語義槽位，讓用戶看到「專業結構」而不是一坨文字。
 * 純函數：按句號/破折號切分短句，正則歸類到槽位；歸不進任何槽的短句
 * 併入主體（用戶的原話幾乎總是主體描述）。
 */
export function extractSixSlots(prompt: string, mediaType: "image" | "video"): SixSlots {
  const clauses = prompt
    .split(/[.。]|\s—\s/)
    .map((clause) => clause.trim())
    .filter(Boolean);
  const slots: SixSlots = { subject: "", scene: "", composition: "", lighting: "", style: "", quality: ""
};
  const patterns =
    mediaType === "image"
      ? { subject: IMAGE_SLOT_PATTERNS.subject, scene: IMAGE_SLOT_PATTERNS.setting, composition:
IMAGE_SLOT_PATTERNS.composition, lighting: IMAGE_SLOT_PATTERNS.lighting, style: IMAGE_SLOT_PATTERNS.style }
      : { subject: /\b(man|woman|cat|dog|person|people|figure|character|he|she|it|they)\b/i, scene:
SLOT_PATTERNS.setting, composition: SLOT_PATTERNS.action, lighting: SLOT_PATTERNS.lighting, style:
SLOT_PATTERNS.style };
```

代码块 6｜确定性节点标识：双轮 FNV-1a 哈希　对应 4.2「同一份声明反复执行结果完全一致……节点标识保持相等」——标识由纯函数哈希生成，同输入始终生成同一 id。
来源：`src/lib/workflow-dsl/compile.ts` L65–85 逐字摘录。

```
function fnv1a32(seed: number, input: string): number {
  let hash = seed >>> 0;
  for (let i = 0; i < input.length; i++) {
    hash ^= input.charCodeAt(i);
    hash = Math.imul(hash, FNV_PRIME);
  }
  return hash >>> 0;
}

function toHex8(value: number): string {
  return value.toString(16).padStart(8, "0");
}

/** wf-${h1.hex}${h2.hex}（wf- + 16 hex）。同輸入永遠同 id。 */
export function workflowNodeId(workflow: string, stepId: string): string {
  const key = `${workflow}/${stepId}`;
  const h1 = fnv1a32(FNV_OFFSET_BASIS, key);
  const h2 = fnv1a32(FNV_OFFSET_BASIS ^ SECOND_ROUND_SEED, key);
  return `wf-${toHex8(h1)}${toHex8(h2)}`;
}
```

代码块 7｜任务队列认领：FOR UPDATE SKIP LOCKED 与租约心跳　对应 3.3「多工作进程并发互不冲突」——行锁跳过已被占用的候选；过期任务也必被认领一次以走终结与退款路径，不产生永久排队且已扣费的任务。
来源：`src/lib/generation-core/job-repository.ts` L273–300 逐字摘录。

```
export async function claimNextGenerationJob(
  workerId: string,
  leaseMs: number,
  sql: GenerationSql = getGenerationSql(),
): Promise<GenerationJob | null> {
  const rows = await sql<Row[]>`
    WITH candidate AS (
      SELECT id
      FROM generation_job
      WHERE status = 'queued'
        AND available_at <= now()
      -- Expired queued jobs must still be claimed once so the worker can move
      -- them to a terminal state and execute the existing refund path. Leaving
      -- them out here creates permanently queued, already-charged tasks.
      ORDER BY (deadline_at <= now()) DESC, available_at, created_at
      FOR UPDATE SKIP LOCKED
      LIMIT 1
    )
    UPDATE generation_job job
    SET status = 'leased', lease_owner = ${workerId},
        lease_expires_at = now() + (${leaseMs}::text || ' milliseconds')::interval,
        heartbeat_at = now(), started_at = COALESCE(started_at, now()), updated_at = now()
    FROM candidate
    WHERE job.id = candidate.id
    RETURNING job.*
  `;
  return rows[0] ? mapGenerationJob(rows[0]) : null;
}
```

代码块 8｜供应商并发租约：三层上限取小与负载均衡排序　对应 3.3「在飞任务数与供应商、账号、能力三层并发上限取最小值」——三层在飞计数各与其并发上限比较；按供给层级、健康度、负载率、延迟与成本排序后 SKIP LOCKED 取一，租约按 job 唯一键抢占防重复派发。
来源：`src/lib/generation-core/scheduler.ts` L63–79、L85–98 逐字摘录。

```
        AND (SELECT count(*) FROM supplier_lease lease
          WHERE lease.supplier_id = supplier.id AND lease.expires_at > now()) < supplier.concurrency_limit
        AND (SELECT count(*) FROM supplier_lease lease
          WHERE lease.supplier_account_id = account.id AND lease.expires_at > now()) <
account.concurrency_limit
        AND (SELECT count(*) FROM supplier_lease lease
          WHERE lease.supplier_capability_id = capability.id AND lease.expires_at > now()) <
capability.concurrency_limit
      ORDER BY CASE supplier.supply_tier
          WHEN 'first_party' THEN 0 WHEN 'managed_pool' THEN 1 WHEN 'relay' THEN 2 ELSE 3 END,
        supplier.priority ASC,
        CASE supplier.health_status WHEN 'healthy' THEN 0 WHEN 'degraded' THEN 1 ELSE 2 END,
        supplier.health_score * supplier.quality_score * capability.health_score DESC,
        (SELECT count(*)::float FROM supplier_lease lease
          WHERE lease.supplier_id = supplier.id AND lease.expires_at > now()) / supplier.concurrency_limit
ASC,
        supplier.latency_ewma_ms ASC NULLS LAST,
        (supplier.unit_cost + capability.cost_units) ASC,
        supplier.last_selected_at ASC NULLS FIRST
      FOR UPDATE OF supplier SKIP LOCKED
// ……（此处摘录略去连续行）
    const leaseId = randomUUID();
    const expiresAt = new Date(Date.now() + input.leaseMs);
    const inserted = await tx<Row[]>`
      INSERT INTO supplier_lease (
        id, supplier_id, supplier_account_id, supplier_capability_id, model_id,
        job_id, worker_id, expires_at
      ) VALUES (
        ${leaseId}, ${String(supplier.id)}, ${String(supplier.account_id)},
${Number(supplier.capability_id)},
        ${input.job.modelId}, ${input.job.id}, ${input.workerId}, ${expiresAt}
      )
      ON CONFLICT (job_id) DO NOTHING
      RETURNING id
    `;
    if (!inserted[0]) return null;
```

代码块 9｜计费操作的状态机认领　对应 3.3「幂等引用＋状态机认领防止重复扣费」——仅 pending 且到期的一条可被 UPDATE 为 executing 并带上 45 秒执行租约；RETURNING 无行即认领失败，不会出现第二个执行者。
来源：`src/lib/generation-core/billing-ledger.ts` L90–112 逐字摘录。

```
async function claimBillingOperation(input: {
  jobId: string;
  operation: BillingOperation;
  executorId: string;
  sql: GenerationSql;
}): Promise<Row | null> {
  const rows = await input.sql<Row[]>`
    UPDATE billing_operation
    SET status = 'executing',
        executor_id = ${input.executorId},
        execution_expires_at = now() + (${EXECUTION_LEASE_MS}::text || ' milliseconds')::interval,
        last_attempt_at = now(),
        attempt_count = attempt_count + 1,
        updated_at = now()
    WHERE job_id = ${input.jobId}
      AND operation = ${input.operation}
      AND sequence = 1
      AND status = 'pending'
      AND next_attempt_at <= now()
    RETURNING *
  `;
  return rows[0] ?? null;
}
```

代码块 10｜记账提交：已生效的预订才允许提交，且只提交一次　对应 3.3「事件账本只追加不修改」与「重放不双记账」——commit 是纯追加且以 JOIN 断言 reserve 已 applied，唯一键冲突静默跳过；RETURNING 仅用于保证「账已结」事件属实且不重复。
来源：`src/lib/generation-core/billing-ledger.ts` L263–297 逐字摘录。

```
/**
 * A commit has no external side effect: it proves how much of an already-applied
 * reservation was consumed. The unique operation key makes this idempotent.
 */
export async function commitGenerationBilling(
  job: GenerationJob,
  amount = job.billingAmount,
  sql: GenerationSql = getGenerationSql(),
): Promise<void> {
  if (!amount) return;
  if (amount < 0 || amount > job.billingAmount) {
    throw new Error(`Invalid committed amount for generation job ${job.id}.`);
  }

  // 埋點需要知道 INSERT 是否真正落地（重複 reconcile 會命中 ON CONFLICT 無聲跳過），
  // 故追加 RETURNING id——不改變寫入語義，只讓「帳已結」事件不撒謊、不重複。
  const applied = await sql<Row[]>`
    INSERT INTO billing_operation (
      id, job_id, operation, amount, status, idempotency_ref, applied_at,
      next_attempt_at
    )
    SELECT ${randomUUID()}, job.id, 'commit', ${amount}, 'applied',
      ${`generation:${job.id}:commit:1`}, now(), now()
    FROM generation_job job
    JOIN billing_operation reserve
      ON reserve.job_id = job.id
      AND reserve.operation = 'reserve'
      AND reserve.sequence = 1
      AND reserve.status = 'applied'
    WHERE job.id = ${job.id}
    ON CONFLICT (job_id, operation, sequence) DO NOTHING
    RETURNING id
  `;
  if (applied[0]) {
    emitJobEvent({ jobId: job.id, event: JOB_EVENT.billingCommitted, actor: "system", detail: { amount,
operation: "commit" } }, sql);
```

代码块 11｜僵尸回收门槛与退款幂等　对应 3.3「超阈值僵尸任务由对账强制回收并全额退款」——退款操作按幂等引用追加，重复触发命中唯一键即跳过；退款金额与既有记录不一致立即改道人工对账，不自动改写。
来源：`src/lib/generation-core/billing-ledger.ts` L18–24、L301–321 逐字摘录。

```
const EXECUTION_LEASE_MS = 45_000;
const DEFAULT_RECONCILE_BATCH = 50;

// P0 止血（2026-08-28 域 E）：殭屍回收門檻。非終態卡在計費狀態超過此時長的任務，
// 由 reconcile 強制對帳（該 refund 的 refund、確定性失敗的終結），不再無限重試。
// P1 拆除 reserve 阻塞閘門後本段整體退役。
const ZOMBIE_RECLAIM_MIN_AGE_MS_DEFAULT = 6 * 60 * 60 * 1000;
// ……（此处摘录略去连续行）
async function ensureRefundOperation(job: GenerationJob, amount: number, sql: GenerationSql): Promise<Row>
{
  await sql`
    INSERT INTO billing_operation (
      id, job_id, operation, amount, status, idempotency_ref, next_attempt_at
    ) VALUES (
      ${randomUUID()}, ${job.id}, 'refund', ${amount}, 'pending',
      ${`generation:${job.id}:refund:1`}, now()
    )
    ON CONFLICT (job_id, operation, sequence) DO NOTHING
  `;
  const rows = await sql<Row[]>`
    SELECT * FROM billing_operation
    WHERE job_id = ${job.id} AND operation = 'refund' AND sequence = 1
    LIMIT 1
  `;
  const operation = rows[0];
  if (!operation) throw new Error(`Could not create refund operation for generation job ${job.id}.`);
  if (operationAmount(operation) !== amount) {
    throw new Error(`Refund amount conflict for generation job ${job.id}; manual reconciliation
required.`);
  }
  return operation;
```

代码块 12｜**网关不可达时的降级回落**　对应 3.3「网关不可达时自动回落降级路径，对外交付规格不变」——不可达判定沿既有预订路径继续放行（ok:true），预检不引入新语义。
来源：`src/lib/generation-core/quota-precheck.ts` L160–170 逐字摘录。

```
  }

  if (precheck.verdict === "unreachable") {
    // 不可達：沿用現有代碼對 new-api 不可用的既有姿勢—回落原 reserve 路徑，
    // 由 billing-ledger 按既有分類凍結/重試，閘門不發明新語義。
    console.error(
      `[quota-precheck] new-api unreachable for biz key ${input.bizKeyId} userId=${input.billingUserId};
falling back to incumbent reserve: ${precheck.reason}`,
    );
    return { ok: true, billingAmount: input.baseBillingAmount };
  }
```

代码块 13｜**事件账本：只追加、上游错误原文入册、不阻塞主流程**　对应 3.3「事件账本只追加不修改，上游错误原文入册」——emitJobEvent 对 job_events 仅 INSERT，fire-and-forget：埋点失败只记日志，遥测可以晚到、不可以不写、不挡主流程。
来源：`src/lib/generation-core/job-events.ts` L3–9、L65–78 逐字摘录。

```
// P1.5 事件賬本 + probe 偵查面（2026-08-28 中控外科手術）。
// 目標：任意一筆失敗 job，probe 一條命令 30 秒內給出死因 + 上游錯誤原文，
// 全程不開 docker exec、不進 VPS。
//
// 遙測兩鐵律：可以晚到，不可以不寫；不擋路。
// - emitJobEvent 是 fire-and-forget：埋點失敗只 console.error，永不讓主流程失敗。
// - 上游錯誤原文（upstreamRaw）原樣入庫不改寫；話術只許存在於展示層。
// ……（此处摘录略去连续行）
 */
export function emitJobEvent(input: JobEventInput, sql: GenerationSql = getGenerationSql()): void {
  const detailJson = normalizeDetail(input.detail);
  const tag = `[job-events] job=${input.jobId} event=${input.event}`;
  try {
    void sql`
      INSERT INTO job_events (job_id, event, actor, upstream_raw, detail)
      VALUES (${input.jobId}, ${input.event}, ${input.actor ?? null}, ${input.upstreamRaw ?? null},
${detailJson}::jsonb)
    `.catch((error: unknown) => {
      console.error(`${tag} emit failed`, error);
    });
  } catch (error) {
    console.error(`${tag} emit failed`, error);
  }
```

代码块 14｜**二次开发其一：工具目录挂载与派遣入口实现审计**　对应 3.3「90 个自研设计工具以框架原生工具目录挂载并接受实现审计」——工具以 opencode 原生 tool 声明形式挂载；派遣入口按 expert 白名单拒绝式稽核，清单不可读时返回 503，未知专家返回 400。

来源：`agent-harness/gateway/subagent.ts` L1–26、L636–663 逐字摘录。

```
import { tool } from "@opencode-ai/plugin/tool"
import { ok, fail } from "../lib/client"

const GATEWAY = process.env.GATEWAY_URL || "http://127.0.0.1:8081"

const POLL_INTERVAL_MS = 4000
const POLL_DEADLINE_MS = 16 * 60 * 1000 // 网关侧 15 分钟硬顶会先以 error 收尾

export default tool({
  description:
    "派一个子代理（subagent）执行一个自包含任务。用于多步骤任务、大素材加工、可并行的探索/生成——把活拆给子代理，主对话保持精简。任务书（prompt）必须自包含：目标、素材路径/数据、验收标准、预期产出格式，四样缺一不可（子代理看不到你们的对话历史，只能拿到 prompt 里的信息）。async=true（后台模式，默认推荐）：立即返回，主对话可继续推进其他工作或结束回合；子代理完成后系统自动唤醒你并附结果摘要（失败也唤醒并附死因），禁止轮询等待。async=false（默认，同步模式）：阻塞等待完成并直接返回 summary，仅用于必须立即拿结果才能继续的小任务。并发上限 3；满了工具会返回「已有 3 个子代理在跑，等一个完成再派」，此时等一个完成再派，不要连续重试。简单单步任务（答一句话、一张图的直接生成）不要派，避免杀鸡用牛刀。野獸模式下派 Expert 域內任務時指定 expert 欄（如 3D 場景→director3d、剪輯→editor），讓子代理以該域專家身分與工具表執行。",
  args: {
    label: tool.schema.string().describe("短标签，简体中文，≤20 字，如「第一章总结」「素材整理」"),
    prompt: tool.schema.string().describe("给子代理的完整任务书（自包含：目标、素材路径/数据、验收标准、预期产出格式）"),
    expert: tool.schema
      .string()
      .optional()
      .describe(
        "Expert id（僅野獸模式生效）：director3d=3D導演 / music=音樂導演 / video-director=視頻導演 / editor=剪輯師 / screenwriter=編劇分鏡 / motion-director=動態設計 / web3d-director=Web3D導演。指定後子代理以該 Expert 身分執行（僅持該域工具）。非野獸模式或省略時=通用子代理。",
      ),
    async: tool.schema
      .boolean()
      .describe(
        "true=后台派出立即返回，完成后自动唤醒主会话附结果摘要（失败也唤醒），军团并发派遣一律用它；缺省/false=同步等待结果（仅限必须立即拿结果的小任务）",
      ),
  },
// ……（此处摘录略去连续行）
  // R8 修復片2（2026-09-10）：Expert 路由輸入端。payload 可選 expert（字串）；
  // 僅 beast_mode ON 時生效（OFF 時收到的 expert 靜默丟棄 = 零 OFF 回歸）。
  // ON 時必須通過 gateway/expert/expert-ids.json 白名單（{"experts":[...]}）校驗：
  // 檔不存在/JSON 壞/形狀錯 → 拒絕（ok:false「expert 清單不可讀」，不裸奔）；
  // expert 不在清單 → 拒絕（ok:false「未知 expert」）。通過才寫進 run.agent。
  let expertId = ""
  const rawExpert = typeof body.expert === "string" ? body.expert.trim() : ""
  if (rawExpert && deps.beastMode(parent)) {
    let allowed: unknown
    try {
      allowed = (JSON.parse(readFileSync(`${import.meta.dir}/expert/expert-ids.json`, "utf8")) as { experts?: unknown })?.experts
    } catch (e) {
      return Response.json(
        { ok: false, error: `expert 清單不可讀（gateway/expert/expert-ids.json：${e instanceof Error ? e.message : String(e)}）` },
        { status: 503 },
      )
    }
    if (!Array.isArray(allowed)) {
      return Response.json(
        { ok: false, error: "expert 清單不可讀（gateway/expert/expert-ids.json 形狀錯誤：缺 experts 陣列）" },
        { status: 503 },
      )
    }
    if (!(allowed as string[]).includes(rawExpert)) {
      return Response.json({ ok: false, error: `未知 expert：${rawExpert}` }, { status: 400 })
    }
    expertId = rawExpert
  }
```

代码块 15｜**二次开发其二：提问中断改造为人工闸门**　对应 3.3「框架原生的提问中断机制改造为创作流程中的人工闸门」——问题登记入待答队列并挂心跳与超时代答计时器；答复按 request_id 校验匹配，未登记时返回 404。
来源：`agent-harness/gateway/index.ts` L4984–5010、L5072–5090 逐字摘录。

```
async function emitQuestionRequest(state: SessionState, callId: string, input: unknown): Promise<void> {
  if (!callId) return
  for (const pending of state.pendingQuestions.values()) {
    if (pending.callId === callId) return
  }
  let lookedUp = await lookupOpencodeQuestion(state, callId)
  for (let attempt = 0; attempt < 8 && !lookedUp?.id; attempt++) {
    await new Promise((resolve) => setTimeout(resolve, 250))
    lookedUp = await lookupOpencodeQuestion(state, callId)
  }
  const fallbackQuestions = normalizeQuestionItems((input as { questions?: unknown } | null)?.questions)
  const requestId = lookedUp?.id ?? ""
  const questions = lookedUp?.questions?.length ? lookedUp.questions : fallbackQuestions
  if (!requestId || questions.length === 0) {
    console.warn(`[gateway] question tool ${callId} has no que_* request yet`)
    if (state.toolStatusSeen.has(`${callId}:running`)) state.toolStatusSeen.delete(`${callId}:running`)
    if (state.toolStatusSeen.has(`${callId}:pending`)) state.toolStatusSeen.delete(`${callId}:pending`)
    return
  }
  if (state.pendingQuestions.has(requestId)) return
  state.pendingQuestions.set(requestId, { requestId, callId, questions })
  emitToSession(state, "question.request", { request_id: requestId, questions })
  // R8-B 工單 #3: pending 期間 30s 心跳接管用戶可見性 (R8 實錘 129s 零幀死寂)。
  armQuestionHeartbeat(state)
  // 片AJ: 3 分鐘無人回覆按推薦方案代答繼續 (assumed)。
  armQuestionAssumeTimer(state)
}
// …… (此处摘录略去连续行)
async function handleQuestion(key: string, id: string, req: Request): Promise<Response> {
  const state = getOwnedSession(key, id)
  if (!state) return jsonError(404, "session not found", "not_found", "not_found")
  let body: { request_id?: string; answers?: unknown }
  try {
    body = (await req.json()) as { request_id?: string; answers?: unknown }
  } catch {
    return jsonError(400, "invalid json body", "bad_request", "bad_request")
  }
  const requestId = typeof body.request_id === "string" ? body.request_id.trim() : ""
  if (!requestId || !requestId.startsWith("que")) {
    return jsonError(400, "request_id is required", "bad_request", "bad_request")
  }
  if (!isQuestionAnswers(body.answers)) {
    return jsonError(400, "answers is required", "bad_request", "bad_request")
  }
  if (!state.pendingQuestions.has(requestId)) {
    return jsonError(404, "question request unknown or already resolved", "not_found", "question_expired")
  }
```

代码块 16｜**二次开发其三：闲置回收改写为惰性恢复**　对应 3.3「闲置回收（idle reaper）改写为数据库存档七天＋重开惰性恢复，用户隔天回到历史会话仍能完整回放」——回收只清内存、行保留（keepRow），重开经 hydrateFromDb 惰性复活；常量块显示默认 30 天且清扫关闭。
来源：`agent-harness/gateway/index.ts` L552–568、L8164–8198、L1078–1106 逐字摘录。

```
const REPLAY_BUFFER_SIZE = 1000
const IMAGE_URL_DEDUP_CAP = 100
const PART_TEXT_TRACK_CAP = 200
const TOOL_STATUS_DEDUP_CAP = 500
const STEP_COST_LOG_CAP = 1000
const MESSAGE_ROLE_CAP = 400
const SUPPRESSED_PART_CAP = 500
// Reaper thresholds. A session with no SSE clients and no activity for this
// long is assumed abandoned (closed tab / lost network) and freed.
// 2026-09-12 永久保留改造：默認 TTL 從 30 分鐘放寬到 30 天。reap（keepRow=true）只清內存、
// 不刪數據也不再發 session.closed（見 teardownSession）；正常使用週期內會話基本不會被 reap，
// 內存熱度回收能力保留。env 為逃生口（毫秒）。
const SESSION_IDLE_TTL_MS = Number.parseInt(process.env.SESSION_IDLE_TTL_MS || "2592000000", 10) ||
2_592_000_000
// 2026-09-12 永久保留：7 天 stale sweep（刪 DB row + opencode 會話 + uploads）默認關閉。
// 未設/0 = 永不刪；>0 恢復舊的「N 天未更新即連根刪」能力（env 逃生口）。
const SESSION_SWEEP_DAYS = Number.parseInt(process.env.SESSION_SWEEP_DAYS || "0", 10) || 0
const SESSION_REAP_INTERVAL_MS = 60_000
// ……（此处摘录略去连续行）
// 200-frame replay buffer — for the lifetime of the process. A session is
// reaped only when it has NO connected SSE clients AND no activity for
// SESSION_IDLE_TTL_MS, so an active stream or an in-flight generate is never
// collected. Since 2026-08-29 the upstream opencode session is NOT deleted
// here: message history lives there, and deleting it turned revived sessions
// into empty shells. Cleanup rides the 7-day stale sweep and explicit DELETE.
setInterval(() => {
  const now = Date.now()
  for (const [ticket, rec] of sseTickets) {
    if (rec.expiresAt <= now) sseTickets.delete(ticket)
  }
  for (const [ticket, rec] of uploadTickets) {
    if (rec.expiresAt <= now) uploadTickets.delete(ticket)
  }
  for (const [id, state] of sessions) {
    // Drop confirm answers past their retention window so a long-lived session
    // does not accumulate them forever.
    for (const [requestId, rec] of state.resolvedConfirms) {
      if (now - rec.resolvedAt > CONFIRM_RESOLVED_TTL_MS) state.resolvedConfirms.delete(requestId)
    }
    if (state.clients.size > 0) continue
    // A pending confirm pins the session. With CONFIRM_TIMEOUT_MS at 15 min and
    // SESSION_IDLE_TTL_MS at 30 min this cannot pin it forever: the confirm
    // expires first, and the next sweep is free to reap.
    if (state.pendingConfirms.size > 0) continue
    if (state.pendingQuestions.size > 0) continue
    if (now - state.lastActivityAt < SESSION_IDLE_TTL_MS) continue
    const idleMs = now - state.lastActivityAt
    teardownSession(id, state, "idle_timeout", true)
    // 2026-08-29 修復「空殼會話」：閒置回收不再刪 opencode 會話與上傳目錄。
    // 訊息歷史權威在 opencode，DB row（keepRow=true）revive 後靠它回填正文；
    // 之前這裡刪掉 opencode → 恢復出來只有殼沒有內容（天天丟記錄）。
    // opencode/上傳的清理只跟 7 天 stale sweep 與顯式 DELETE 走。
    console.log(`[gateway] reaped idle session ${id} after ${Math.round(idleMs / 1000)}s idle`)
  }
// ……（此处摘录略去连续行）
// 內存 miss 時從 DB lazy reload（進程重啟後的會話恢復路徑）。
// 選 lazy 而非啟動全量載入：冷會話佔多數，lazy 免付啟動成本、不把殭屍
// session 釘在內存，且 reaper 的 idle 規則照常生效。
function hydrateFromDb(id: string): SessionState | null {  const row = loadSessionById(id)
  if (!row) return null
  const state = hydrateRow(row) as unknown as SessionState
  // 0831: hydrateRow（persistence.ts，本檔不改）不認得 sessionTasks，重啟恢復的 session
  // 該欄為 undefined；此處補空 Map，令所有下游 state.sessionTasks.* 存取安全（在途閘自然放行）。
  if (!state.sessionTasks) state.sessionTasks = new Map()
  sessions.set(state.sessionId, state)
  // 畫布快照回灌：DB 鏡像 → canvasStates，重啟後 /internal/canvas/snapshot
  // 仍能讀到完整節點/連線（TTL 重新起算，行為等同活躍畫布）。
  if (state.canvasSnapshot && typeof state.canvasSnapshot === "object") {
    canvasStates.set(state.sessionId, { state: state.canvasSnapshot, updatedAt: Date.now() })
  }
  // 工作流快照回灌（對齊 canvas 快照）：DB 鏡像 → workflowService，重啟後
  // /internal/workflow/status 仍能讀到該工作流的真實狀態。
  try {
```

```
    restoreWorkflowsFromSnapshot(state)
  } catch (e) {
    console.error(`[gateway] workflow restore failed for ${state.sessionId}:`, e)
  }
  console.log(`[gateway] session restored from db: ${state.sessionId} (opencode
${state.opencodeSessionId})`)
  maybeReprompInterruptedTurn(state)
  return state
}

function getOwnedSession(key: string, id: string): SessionState | null {
  const state = sessions.get(id) ?? hydrateFromDb(id)
```

代码块 17｜**二次开发其四：停止按钮转发会话中止端点**　对应 3.3「停止按钮转发框架的会话中止端点，当前轮次硬停而非软停」——网关代理上游 abort 并置 turnAborted 硬停本回合。
来源：`agent-harness/gateway/index.ts` L5466–5486 逐字摘录。

```
async function handleAbortSession(key: string, id: string): Promise<Response> {
  const state = getOwnedSession(key, id)
  if (!state) return jsonError(404, "session not found", "not_found", "not_found")
  try {
    const res = await fetch(`${OPENCODE_URL}/session/${state.opencodeSessionId}/abort`, {
      method: "POST",
      headers: opencodeHeaders(),
      signal: AbortSignal.timeout(5000),
    })
    if (!res.ok) {
      return jsonError(502, `opencode abort failed: ${res.status}`, "backend_error", "backend_error")
    }
  } catch {
    return jsonError(502, "opencode backend unreachable", "backend_unreachable", "backend_error")
  }
  state.turnAborted = true // 用戶主動中止：不觸發抖動重試
  // N2 同款（2026-09-02）：用戶中止即剝掉本輪文本幀—SSE 斷線重連不再整段
  // 重放被中止輪的舊文案（stale delta 閃回根治）。
  stripStaleTurnTextFromReplay(state)
  return Response.json({ ok: true })
}
```

代码块 18｜**多专家路由与能力断路**　对应题名与摘要「多专家智能体协同」——编排者按开关注入 agent 路由字段并物理裁剪工具表：orchestrator 名下 24 项设计工具逐条置 false，越权工具在运行时不存在，而非依靠提示词劝阻。
来源：`agent-harness/workspace/.opencode/opencode.json` L936–950、L4602–4635、L201–233 逐字摘录。

```
}

// ── 能力斷路（MOE 評測死線1，2026-09-08）：野獸模式 ON 時主會話跑 orchestrator
// agent（opencode.json agent.orchestrator.tools 把 blender_exec/blender_render/
// blender_export_gltf/music_gen 從工具表物理移除）→ 主會話無法直調域工具，只能
// subagent_dispatch 派發 Expert。OFF 不帶 agent 欄＝default build agent＝零變化。
function beastAgentField(state: SessionState): Record<string, string> {
  try {
    if (loadUserSettings(state.apiKey)?.beast_mode === true) return { agent: "orchestrator" }
  } catch {}
  return {}
}

// ── N1 規格3（2026-08-27）：重啟斬 turn 的自動重投 ──────────────────
// 評測實錘：opencode-server 重啟斬殺 in-flight turn（無 idle、無 turn.done），
// ……（此处摘录略去连续行）
        // R8 修復片2（2026-09-10）：能力斷路·物理事實──R7 狀態聲明之上的補丁句
        //  （R7 遺留#2 的宣告升級）。orchestrator agent 工具表已物理裁剪
        //  （opencode.json agent.orchestrator.tools），digest 的「能力斷路」聲明
        // 由此升級為物理事實；subagent_dispatch（帶 expert 欄）是域工具唯一通路。
        const capabilityCutoffFact =
          "【能力斷路·物理事實】上列註冊表宣告的域工具不在你的工具表裡（orchestrator agent 工具表已物理裁剪，轉移調用會直接失敗）。經 subagent_dispatch（帶 expert 欄）派遣對應 Expert 是執行它們的唯一通路。AGENTS.md 路由表中對應這些工具的行，由你派遣的 Expert 代為執行；其餘工具（image_generate 等）的直調路由仍按 AGENTS.md 執行。"
        // R5 修復片（2026-09-11）：優先序聲明──野獸幀＋派遣手冊優先於 workspace/AGENTS.md，
        // 消除「你負責擴寫／親自操作 workflow_*」（AGENTS.md）vs「你本人不產出任何內容」
        //  （派遣手冊第〇節）的語義分歧（R8 審計 E7；1204 秒零工具調用病灶實證）。
        const beastPrecedenceFact =
          "【優先序·野獸模式】本幀與《派遣手冊》優先於 workspace/AGENTS.md。AGENTS.md 中「你負責擴寫／你親自寫 prompt／你親自操作 workflow_* 工具」等指示，在野獸模式下由你派遣的 Expert 執行；內容生產（寫分鏡 prompt、生成圖片影片）你一律不親手做。你的動作只有：拆解任務 → subagent_dispatch（必帶 expert 欄，async:true 為常態）→ 派出後立刻用一句正文向用戶簡報 → 收完工通知後匯總回報。AGENTS.md 的安全紅線與文案規範繼續有效。"
        if (!playbookBody || !digestBody) {
          return jsonError(
            503,
            `野獸模式已開但派遣手冊或註冊表摘要不可讀（gateway/expert/）：${frameError || "檔案為空"}。拒絕裸奔主prompt，请修复部署或暂时关闭 beast_mode。`,
            "beast_frame_unavailable",
            "beast_frame_unavailable",
          )
        }
        promptBody.system = [
          "【野獸模式：已啟用】以下派遣手冊與 Expert 註冊表由系統注入，這是當前系統的真實狀態：BEAST_MODE 現在就是 ON，以下全部條款立即生效，無需質疑、無需再確認任何 gate 條件。",
          playbookBody,
          digestBody,
          beastPrecedenceFact,
          capabilityCutoffFact,
          "---",
          promptBody.system,
        ]
          .filter(Boolean)
          .join("\n\n")
        // 能力斷路：ON 輪主會話掛 orchestrator agent（域工具已從工具表物理移除，
        // playbook prompt 聲明＋工具表移除雙保險；見 opencode.json agent.orchestrator）。
        promptBody.agent = "orchestrator"
      }
// ……（此处摘录略去连续行）
  "agent": {
    "orchestrator": {
      "mode": "all",
      "description": "MOE Orchestrator（野獸模式主會話）：域工具已斷路，只能 subagent_dispatch 派發 Expert",
      "tools": {
        "blender_exec": false,
        "blender_export_gltf": false,
        "blender_render": false,
        "image_quote": false,
        "music_gen": false,
        "video_quote": false,
        "video_upload": false,
        "last_frame_video_chain": false,
        "timeline_ops": false,
        "timeline_query": false,
```

```
        "timeline_rhythm": false,
        "timeline_style": false,
        "timeline_mix": false,
        "timeline_master": false,
        "timeline_lint": false,
        "timeline_restore": false,
        "editing_manifest": false,
        "audio_first_timeline": false,
        "script_scene_parse": false,
        "storyboard_qc_17": false,
        "story_sequence_consistency": false,
        "canvas_apply_ops": false,
        "doc_struct": false,
        "world_bible_extract": false,
        "keyframe_render": false,
        "question": true
      }
    },
```

代码块 19｜网关侧硬截止与七类看门狗　对应 3.6「系统在网关侧加入硬截止与看门狗机制」「另有七类看门狗机制避免会话长时间无响应」——常量表共七项；子代理同步派遣按预计时长推导硬截止，超阈转异步放行，不终止在跑任务。
来源：`agent-harness/gateway/subagent.ts` L286–316、L724–749 逐字摘录。

```
// R2-A 看門狗：prompt_async 送達後 TURN_WATCHDOG_MS 內無 tool.invoke 且無
// message.delta = turn 沒啟動（評測：295s 零幀）。先誠實告警再自動重投一次；
// 再等一輪仍無動靜則判會話損壞，建議新開。環境變量可覆蓋（CI 可調短）。
const TURN_WATCHDOG_MS = Number.parseInt(process.env.TURN_WATCHDOG_MS || "90000", 10) || 90_000
// R4-A S3①：上游錯誤（session.error 非 tool_error）後的停滯看門狗。R4 實錘：
// compaction+APIError "Input token exceed" 後 turn 空轉 5 分鐘零告警——啟動
// 看門狗因 turn 已有動靜（turnHasActivity）靜默退出，覆蓋不到「中途死亡」。
// 錯誤發生 90s 內 turn 未收尾且無進度 → 告警 + 依抖動重試兜底；再 90s 仍停滯
// → 明確收尾話術 + turn.done。環境變量可覆蓋。
const TURN_ERROR_WATCHDOG_MS = Number.parseInt(process.env.TURN_ERROR_WATCHDOG_MS || "90000", 10) || 90_000
// N1 規格1：心跳提示延遲——turn 開始後 HEARTBEAT_DELAY_MS 仍無任何
// message.delta 也無 tool.invoke（模型連字都沒吐）時補一條「仍在等待模型响应」。
// 與 S6 ack（30s「收到，正在处理」）錯峰：ack 已發則心跳順延到 45s；每 turn
// 最多 1 條，不刷屏。watchdog（90s）之前的中段窗口由這條填補。
const HEARTBEAT_DELAY_MS = Number.parseInt(process.env.HEARTBEAT_DELAY_MS || "30000", 10) || 30_000
const HEARTBEAT_DELAY_AFTER_ACK_MS = Number.parseInt(process.env.HEARTBEAT_DELAY_AFTER_ACK_MS || "45000",
10) || 45_000
// R5-C（2026-08-28）中段停滯哨：turn 已啟動（有過 tool/正文活動）之後，
// MIDTURN_STALL_MS 內零任何新幀（連進度幀都沒有）= 中段凍結。誠實告警 +
// 防風暴單次重投（復用 watchdogRetried 模式）；重投後仍零幀 → 如實收尾 +
// turn.done。時間戳 turnLastFrameAt 由 emitToSession 全量刷新——長工具執行
// 只要有任何幀（tool.result/canvas/文本）就不誤殺，只咬連幀都沒有的凍結。
const MIDTURN_STALL_MS = Number.parseInt(process.env.MIDTURN_STALL_MS || "90000", 10) || 90_000
// R8-B 工單 #3（2026-08-29）：question pending 心跳週期——pending 期間每 30s
// 一幀用戶可見心跳（R8 實錘：question 卡後 129s 零幀死寂）。
const QUESTION_HEARTBEAT_MS = Number.parseInt(process.env.QUESTION_HEARTBEAT_MS || "30000", 10) || 30_000
// 片AJ（2026-09-07）：question 卡 3 分鐘無人回覆 → 按推薦方案（各題第一選項）代答繼續，
// 答覆文案標注 assumed——無人值守跑批不再掛死（T4 實錘 question 卡挂死 15+ 分鐘）。
const QUESTION_ASSUME_MS = Number.parseInt(process.env.QUESTION_ASSUME_MS || "180000", 10) || 180_000
// R8-B 工單 #5：中段靜默通知閾值——turn 進行中 >45s 無正文（message.delta）
// 補一幀用戶可見進度，此後每 45s 續發。與 R5-C 中段停滯哨（90s 零「任何」幀）
// 並存：停滯哨管「全幀死亡」，本通知管「模型活著但用戶看不見」（長推理段）。
// ……（此处摘录略去连续行）
  if (!asyncMode) {
    // R1（2026-09-11）：同步 dispatch 硬 deadline。到點先讓主會話解鎖（工具輪詢
    // /internal/subagent/status 讀 completed 即取件，實測 <30s 收到 ok:false/TIMEOUT），
    // **子代理 run 不殺**：翻 asyncMode 讓它後台續跑，完成後走既有 wakeParent 回填。
    const deadlineMs = parseSyncDeadlineMs(run.prompt)
    // P2b：arm 時刻記一條，deadline 解析結果可被日誌直接觀測（V3 不必等 240s）。
    console.log(
      `[gateway] subagent sync deadline armed: id=${run.id} label="${run.label}"
deadline=${Math.round(deadlineMs / 1000)}s prompt_head="${run.prompt.slice(0, 24).replace(/\n/g, "\\n")}"`,
    )
    run.deadlineTimer = setTimeout(() => {
      run.deadlineTimer = undefined
      if (run.status !== "running") return
      if (completed.has(run.id)) return
      if (typeof deps?.wakeParent === "function") run.asyncMode = true
      const human =
        deadlineMs < 60_000
          ? `子代理超过预定时间（${Math.round(deadlineMs / 1000)} 秒）未回，已放它后台续跑；你先看其他结果`
          : `子代理超过预定时间（${Math.round(deadlineMs / 60_000)} 分钟）未回，已放它后台续跑；你先看其他结果`
      completed.set(run.id, {
        status: "error",
        summary: "",
        error: human,
        finishedAt: Date.now(),
        label: run.label,
        partial: true,
        reason: "timeout",
```

代码块 20｜报价与判活端点强制同源　对应 3.6「报价与判活端点强制同源」——报价工具直读平台判活端点，可用性是唯一可推荐依据；网关缺省选模型同样只从 available 集合按近 24 小时成功率排序。

来源：`agent-harness/gateway/index.ts` L20–45、L1589–1606 逐字摘录。

```
      const list = body?.data || body?.models || (Array.isArray(body) ? body : [])
      // 判活源（/v1/model-status）：availability 是唯一可推薦依據，靜態目錄不分死活。
      const statusResp = await platformFetch("/model-status", {}, context.sessionID,
context.abort).catch(() => null)
      const statusBody = statusResp && statusResp.ok ? await statusResp.json().catch(() => ({})) : {}
      const statusList: any[] = statusBody?.models ?? []
      const availabilityById = new Map<string, { availability: string; note: string | null }>()
      for (const s of statusList) {
        if (s?.id) availabilityById.set(s.id, { availability: s.availability ?? "unknown", note: s.note ??
null })
      }
      const models = list
        .map((m: any) => ({
          id: m?.id,
          name: m?.name || m?.id,
          coins_per_second: m?.x_coins_per_second ?? null,
          capabilities: m?.x_capabilities ?? m?.capabilities ?? m?.supported_tasks ?? null,
        }))
        .filter((m: any) => m?.id)
        // 影片報價工具只列影片模型：非 video 一律剔除（含 args.model 指定圖片模型時回空，
        // 讓 agent 知道該模型不是影片模型；圖片報價請用 image_quote）。
        .filter((m: any) => m.capabilities?.mediaType === "video")
        .filter((m: any) => (args.model ? m.id === args.model : true))
        .map((m: any) => ({
          ...m,
          resolutions: m.capabilities?.resolutions ?? [],
          durations: m.capabilities?.durations ?? [],
          audio: m.capabilities?.audio ?? false,
// ……（此处摘录略去连续行）
  // R8（2026-09-10 多鏡展開修復）：可用視頻模型動態選型掛點——大片 shotN_gen 不再
  // 寫死 minimax-h3（當前不可用），start 時查 /v1/model-status 選可用視頻模型
  //（輪子 = resolveActiveModel 同源判活；挑選邏輯與其 failover pool 同構：
  // media_type=video + available，按 24h 成功率排序取最優）。選不出回 null
  //（service 側回退引擎默認，submit 前仍有 F2 failover 兜底）。
  pickDefaultVideoModel: async (): Promise<string | null> => {
    const session = stateForWorkflow
    const apiKey = session?.apiKey
    if (!apiKey) return null
    const models = await fetchModelStatusCached(apiKey)
    if (!models) return null
    const pool = (models as Array<{ id: string; media_type?: string; availability?: string; last_24h?: {
success_rate?: number } }>)
      .filter((m) => m.media_type === "video" && m.availability === "available")
    if (pool.length === 0) return null
    pool.sort((a, b) => (b.last_24h?.success_rate ?? 0.5) - (a.last_24h?.success_rate ?? 0.5))
    console.log(`[gateway] R8 default video model picked: ${pool[0].id} (available video
pool=${pool.length})`)
    return pool[0].id
  },
```

代码块 21｜历史回放的传输瘦身与事件游标　对应 4.1「可回放、可交接」——历史响应按前端消费白名单瘦身并 gzip 传输；回放带事件游标与起点标识，重建不重不漏。
来源：`agent-harness/gateway/index.ts` L5316–5345、L5397–5424 逐字摘录。

```
// DB lazy hydrate，reaped/重啟後的 row 可讀)。返回帶游標的物件（前端契約)：
//   { messages: <opencode 消息瘦身陣列（同構 [{info,parts}]，欄位白名單）>, event_counter: <當前計數器>,
//     replay_first_id: <replayBuffer 最舊幀 id，無緩衝則 null> }
// event_counter 給前端做 SSE 續傳游標：restore 全史後 openConnection(counter) 只接增量。
// 上游不可達/非 2xx/會話無 messages → 200 且附 fallback:"replay_only"（前端退回回放兜底)。
// 瘦身：前端唯一消費者 normalizeOpenCodeMessages 只讀 info.role + text/reasoning/tool/file
// 四类 part 的少數欄位（實測 tool.state.output/metadata 與 info 冗餘欄位佔 payload ~80%)，
// 白名單外欄位一率刪除；輸出保持 [{info,parts}] 同構契約，未知 part 型別整塊丟棄。
// 防呆：讀路徑寧跳不炸—任何單則/單 part 處理異常只丟棄該項，函數永不拋錯。
function slimHistoryMessages(msgs: unknown[]): unknown[] {
  const out: unknown[] = []
  for (const rawMsg of msgs) {
    try {
      if (rawMsg === null || typeof rawMsg !== "object" || Array.isArray(rawMsg)) continue
      const msg = rawMsg as Record<string, unknown>
      const infoSrc = msg.info !== null && typeof msg.info === "object" && !Array.isArray(msg.info)
        ? (msg.info as Record<string, unknown>)
        : null
      const info: Record<string, unknown> = {}
      if (infoSrc) {
        if (typeof infoSrc.id === "string") info.id = infoSrc.id
        if (typeof infoSrc.role === "string") info.role = infoSrc.role
      }
      const partsSrc = Array.isArray(msg.parts) ? (msg.parts as unknown[]) : []
      const parts: unknown[] = []
      for (const rawPart of partsSrc) {
        try {
          if (rawPart === null || typeof rawPart !== "object" || Array.isArray(rawPart)) continue
          const part = rawPart as Record<string, unknown>
          const type = part.type
// ……（此处摘录略去连续行）
async function handleGetSessionMessages(key: string, id: string): Promise<Response> {
  const state = getOwnedSession(key, id)
  if (!state) return jsonError(404, "session not found", "not_found", "not_found")
  const cursor = {
    event_counter: state.eventCounter,
    replay_first_id: state.replayBuffer.length > 0 ? state.replayBuffer[0].id : null,
  }
  try {
    const res = await fetch(`${OPENCODE_URL}/session/${state.opencodeSessionId}/message`, {
      headers: opencodeHeaders(),
      signal: AbortSignal.timeout(30_000),
    })
    if (!res.ok) {
      return Response.json({ messages: [], ...cursor, fallback: "replay_only" })
    }
    const msgs: unknown = await res.json()
    if (!Array.isArray(msgs) || msgs.length === 0) {
      return Response.json({ messages: [], ...cursor, fallback: "replay_only" })
    }
    // 成功路徑體積大（實測單會話 452KB)，瘦身後再 gzip 傳輸；降級路徑體積小維持原樣。
    const payload = { messages: slimHistoryMessages(msgs), ...cursor }
    return new Response(gzipSync(Buffer.from(JSON.stringify(payload))), {
      headers: { "Content-Type": "application/json", "Content-Encoding": "gzip" },
    })
  } catch {
    return Response.json({ messages: [], ...cursor, fallback: "replay_only" })
  }
}
```

代码块 22｜交付诚实度：应得镜头与实际落账逐一对账　对应 5.4「确定性门禁与交付规格」在执行侧的对应——引擎按预期镜头集合逐项核对落账结果，宣告四个实得一个即整体判未齐，排除「已提交即完成」的误判。
来源：`agent-harness/gateway/workflow-service.ts` L820–845 逐字摘录。

```
  status(workflowId: string) {
    const wf = this.workflows.get(workflowId)
    if (!wf) return null
    // R10（2026-08-31）交付誠實：final_cut（film 節點）存在時，統計其上游 video
    // 節點的宣告數 vs 實際成功數，暴露「execution success 但 delivery 不完整」
    // 這類假成功（本次事故：宣告4鏡頭實得1鏡頭，status 仍回 success）。不改變既有
    // status 三態語義（execution 維度），只新增只讀統計，向後兼容。
    const filmNodes = wf.graph.nodes.filter((n) => n.kind === "film")
    const delivery = filmNodes.map((filmNode) => {
      const upstreamVideoIds = wf.graph.connections.filter((c) => c.to === filmNode.id).map((c) => c.from)
      const upstreamVideoNodes = wf.graph.nodes.filter((n) => upstreamVideoIds.includes(n.id) && n.kind ===
"video")
      const expectedShots = upstreamVideoNodes.length
      const actualShots = upstreamVideoNodes.filter((n) => wf.nodeStatus[n.id] === "success").length
      const expectedDuration = upstreamVideoNodes.reduce((sum, n) => {
        const d = n.metadata.duration_seconds
        return sum + (typeof d === "number" && d > 0 ? d : 0)
      }, 0)
      return {
        film_node_id: filmNode.id,
        expected_shots: expectedShots,
        actual_shots: actualShots,
        expected_duration_seconds: expectedDuration,
        film_status: wf.nodeStatus[filmNode.id],
        delivery_complete: actualShots === expectedShots && wf.nodeStatus[filmNode.id] === "success",
      }
    })
```

代码块 23｜具名测试断言本体：图与声明往返不变量　对应 7.2「用具名测试断言逐项锁定」——往返不变量测试原文（含断言消息），7.2 表所引即此文件；测试文案为源码原样繁体，不代译。
来源：`src/lib/canvas/layout.test.ts` L342–369 逐字摘录。

```
test("toCanvasProject / fromCanvasProject 往返：座標原樣、units 清空、懸空連線丟掉", () => {
  const n1 = imageNode("n1", 123, 456, { status: "success" });
  const n2: CanvasNode = { ...imageNode("n2", 1000, 2000), type: CanvasNodeType.Text, title: "文本" };
  const state: CanvasReducerState = {
    units: [{ unitId: "cu_1", tool: "image_generate", nodes: [n1, n2], connections: [] }],
    nodes: [n1, n2],
    connections: [
      { id: "c1", fromNodeId: "n1", toNodeId: "n2" },
      { id: "c2", fromNodeId: "n1", toNodeId: "ghost" }, // 懸空：ghost 節點不存在
    ],
  };
  const vp: CanvasViewport = { x: -40, y: 12, k: 1.5 };
  const project = toCanvasProject({ id: "proj-1", title: "我的畫布" }, state, vp, "dots", { totalCoins: 100,
usedCoins: 30 });

  assert.equal(project.id, "proj-1");
  assert.equal(project.title, "我的畫布");
  assert.deepEqual(project.viewport, vp);
  assert.equal(project.backgroundMode, "dots");
  assert.deepEqual(project.budget, { totalCoins: 100, usedCoins: 30 });

  const back = fromCanvasProject(project);
  assert.equal(back.units.length, 0, "單元跨刷新沒有意義，一律清空");
  assert.equal(back.nodes.length, 2);
  assert.deepEqual(back.nodes.find((n) => n.id === "n1")?.position, { x: 123, y: 456 }, "座標要原樣還原，不得重
新排版");
  assert.deepEqual(back.nodes.find((n) => n.id === "n2")?.position, { x: 1000, y: 2000 });
  assert.equal(back.connections.length, 1, "懸空連線要被丟掉");
  assert.equal(back.connections[0].id, "c1");
});
```

代码块 24｜引擎侧幂等守卫：重复启动复用既有工作流　对应 7.2「幂等重放」——工作流启动带幂等守卫，重复调用复用并返回「请勿再次调用」，避免重启重投造成画布卡片堆积。
来源：`agent-harness/gateway/index.ts` L2527–2548 逐字摘录。

```
  if (pathname.endsWith("/workflow/start") && req.method === "POST") {
    const templateId = typeof body.template_id === "string" ? body.template_id : (typeof
url.searchParams.get("template_id") === "string" ? url.searchParams.get("template_id") : null)
    // P0 idempotent workflow_start (2026-08-31 卡片堆積事故): gateway 重啟 →
    // N1 redrive 重投上一條 prompt → agent 再次呼叫 workflow_start → 舊行為會
    // 新建第二套全新 id 的節點堆上畫布 (大量卡片重疊 + 連線亂麻 + 整屏閃爍)。
    // 本檔既有設計已聲明「單 session 只鏡像最新工作流, 反覆 start 屬異常路徑」,
    // 這裡把它落成硬守衛: 同 session 已有同模板且未 success 的工作流 → 復用,
    // 不新建、不重發節點、不發歡迎詞; agent 據 reused/note 繼續推進或 rerun。
    const ownedWfId = workflowOwnerBySession.get(state.sessionId)
    if (ownedWfId && templateId) {
      const owned = workflowService.get(ownedWfId)
      if (owned && owned.templateId === templateId && owned.status !== "success") {
        console.log(`[gateway] workflow_start reused: session=${state.sessionId} wf=${ownedWfId}
status=${owned.status} (P0 idempotent guard)`)
        return Response.json({
          ok: true,
          workflowId: ownedWfId,
          reused: true,
          status: owned.status,
          note: `该会话已存在同模板工作流 (workflow_id=${ownedWfId}, status=${owned.status}), 本次调用已复用, 未新建
任何节点。请勿再次调用 workflow_start; 失败节点用 workflow_rerun 重跑, 推进生成用 workflow_generate。`,
        })
      }
    }
```

代码块 25｜模板逐镜自动展开：一格声明展开为整条镜头链　对应 5.5「自动分镜、逐镜生成、按时间线拼接」的引擎侧支撑——检测镜头角色位后按模板克隆展开 shot2..N 的节点与依赖，旧版静默丢弃多格声明的缺陷就此消除。
来源：`agent-harness/gateway/workflow-engine.ts` L36–70 逐字摘录。

```
/**
 * R8 (2026-09-10 多鏡展開修復): 從 content 抽出 shotN 角色 (N≥2) 的鏡頭編號清單。
 * 大片模板只定義 shot1 鏈; agent 把 shot1..shot8 全部塞進 content 時, 舊版
 * expandTemplate 靜默丟棄 shot2..N → expected_shots=1, agent 被迫手動 add_node
 * 克隆 21 顆節點。這裡把「模板鏈 + content 鏡頭」對齊: content 有 shotN 就展開 N 鏈。
 */
export function detectShotRoles(template: { nodes: { role: string; kind: WorkflowNodeKind }[] }, content:
TemplateContent): string[] {
  // 模板中存在的 shot 角色 (如 shot1)
  const templateShotRoles = template.nodes.filter((n) => /^shot\d+$/.test(n.role)).map((n) => n.role)
  // content 中出現的 shotN 角色 (含模板已定義與 N≥2 新鏡)
  const contentShotRoles = Object.keys(content).filter((k) => /^shot\d+$/.test(k) && content[k] && typeof
content[k] === "object")
  const all = new Set([...templateShotRoles, ...contentShotRoles])
  return [...all].sort((a, b) => (parseInt(a.slice(4), 10) || 0) - (parseInt(b.slice(4), 10) || 0))
}

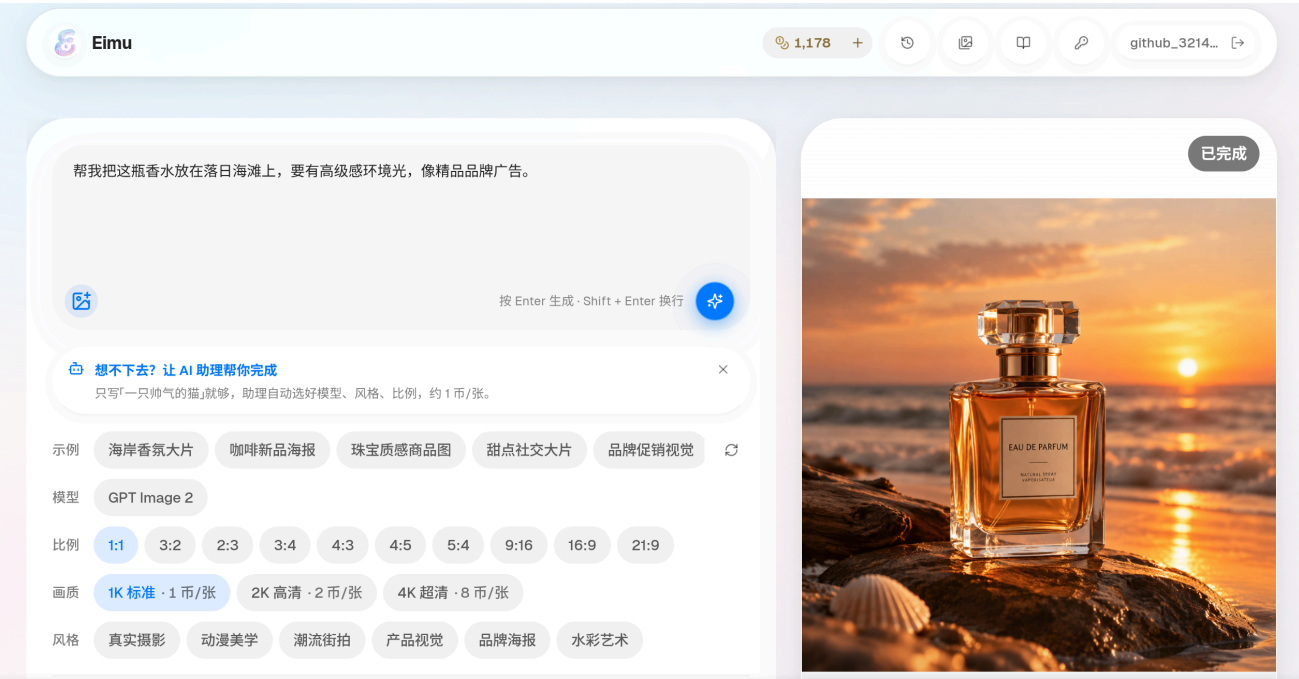
/**
 * 展開模板: 把模板節點實例化成具體節點。
 * role 出現在 contentOverrides 時, merge 進 metadata (Deepseek 的 prompt/model 等)。
 * multiShot 模板: 可由 Deepseek 透過 add_node 追加同 role 的新實例 (id 加 -N 後綴)。
 *
 * R8 (2026-09-10): multiShot 模板 + content 含 shotN (N≥2) 時自動展開 N 條鏈——
 * 以模板 shot1 鏈 (shot1 / shot1_gen / shot1_video) 為原型克隆 shotN 鏈:
 *   shotN → shotN_gen → shotN_video → final_cut 全連,
 *   char_view1 / scene_img 參考連接與 shot1_gen 同構。
 * 暴君裁定 (2026-09-10):「垃圾後端沒執行好, 後端要改」——推翻 08-31 SKILL 的
 * 「agent 手動 add_node 克隆」舊範式, 多鏈展開是後端責任。
 */
export function expandTemplate(
  templateId: string,
  template: {
    id: string
    nodes: { role: string; kind: WorkflowNodeKind; title: string; defaults?: Record<string, unknown> }[]
    connections: { fromRole: string; toRole: string }[]
    multiShot?: boolean
  },
```

附录 B　平台规模与界面证据

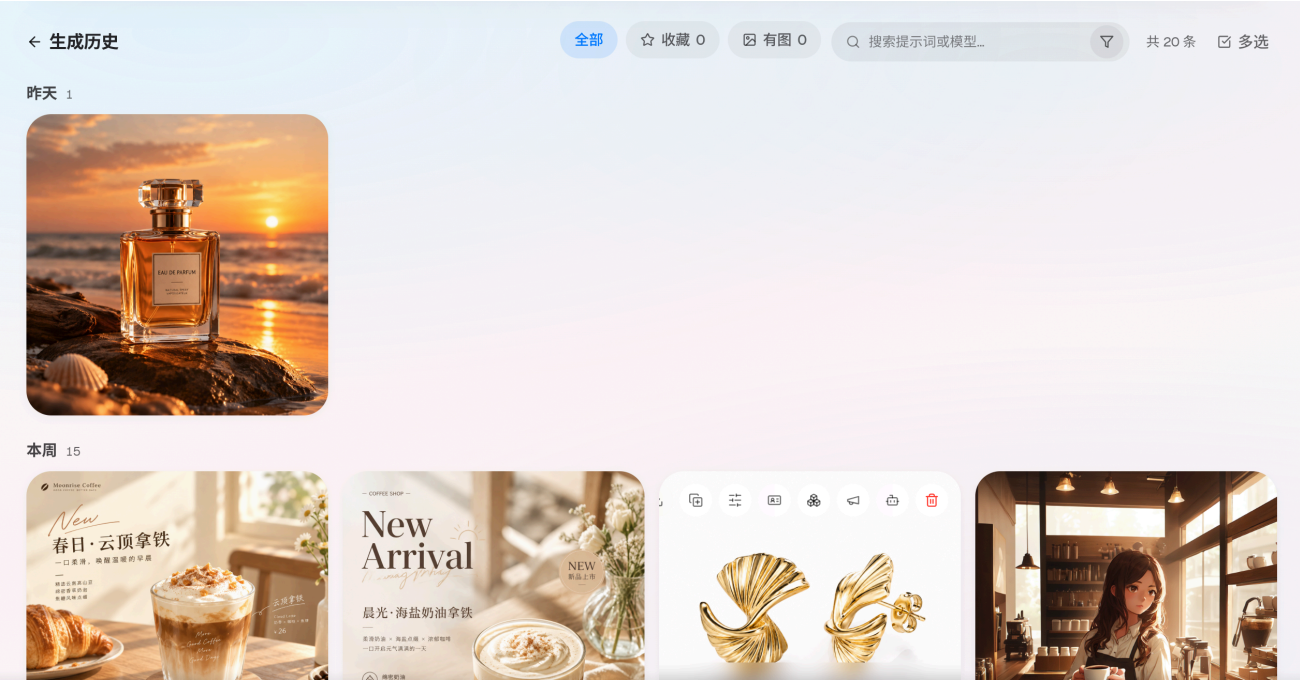
以下材料为平台规模与 5.6 节所述生成工作台、自动归档功能的补充证据。

附表 1　平台规模（截至 2026 年 9 月）

指标	数量	说明
可调用模型 API	56	图像 32 / 视频 23 / 音频 1
AI 可调用工具	90	覆盖生成、剪辑、配乐、资产与质量检验；以 opencode 工具目录挂载并经实现审计（见 3.3）
专业角色	7	各带交付规则与质检要求
领域细分专业资料	617	7 个技术域，含可执行脚本与模板
风格库 / 场景模板 / 镜头要素配方	106 / 14 / 10	起手资产，用户不必从空白开始
创作图节点类型	12	生成类、资产类、组织类三类角色
分镜质检门禁	17	确定性判定，非概率性评价
常用输出栏位	13	一稿多版位批量改写
源码规模	174,218 行	TypeScript，876 个源文件
后端接口 / 界面组件	154 / 121	含开放接口与完整工作台
自动化测试	1816	具名断言锁定表 9 各失败模式（202 个测试文件，全量执行全绿）



附图 1　生图工作台：需求输入、参数选择与结果呈现收束于同一界面，参数与费用在界面可见。



附图 2　生成历史：结果自动归档，按时间分组，支持检索、筛选与多选。

参考文献

[1] J. Adams. *Letter to Abigail Adams*, 12 May 1780. In C. F. Adams (ed.), *Letters of John Adams, Addressed to His Wife*. Boston: Little & Brown, 1841.

[2] E. Horvitz. *Principles of Mixed-Initiative User Interfaces*. Proceedings of ACM CHI, 1999: 159–166.

[3] Anthropic. *Building Effective Agents*. Anthropic Engineering, December 2024.（工作流与智能体二分的经典文献：workflow 由预定义代码掌控控制流，agent 由模型自行决定下一步）

[4] Anthropic. *Effective Context Engineering for AI Agents*. Anthropic Engineering, 2025.（上下文工程作为提示词工程的演进）

[5] M. Hashimoto. *My AI Adoption Journey*. mitchellh.com, 2026-02-05.（harness 工程一词与公式 Agent = Model + Harness 的出处；「智能体每错一次，就把环境改成它再也错不了那一次」亦出此篇）

[6] A. Karpathy. *Vibe coding*（2025 年 2 月）与 *agentic engineering*（2026 年 2 月）. 公开评论。

[7] B. Shneiderman. *Direct Manipulation: A Step Beyond Programming Languages*. IEEE Computer, 1983, 16(8): 57–69.

[8] G. Kahn. *The Semantics of a Simple Language for Parallel Programming*. Proceedings of the IFIP TC2 Working Conference on Programming Systems, North-Holland, 1974: 471–480.（数据流图的数学奠基：节点即变换、边即令牌流，创作图的拓扑调度与此同构）

[9] GitHub. *Spec Kit: Toolkit for Spec-Driven Development (SDD)*. github.com/github/spec-kit, 2025.（规格驱动开发与本平台「先出 IR 再落图」分层的业界对应）

[10] Apple Inc. *Meet Liquid Glass*. WWDC25 Session 219, 2025.

[11] Blender Foundation. *Blender Python API Reference*. Blender 5.2 Documentation, 2026.

[12] FFmpeg Project. *FFmpeg — Multimedia SDK: libavcodec, libavfilter, ffmpeg CLI*. ffmpeg.org, LGPL/GPL 双许可发行.（多镜头拼接、音轨混录与媒体探测所用工具链）

[13] Anomaly（opencode 开发团队）. *opencode: Open-Source AI Coding Agent*. github.com/anomalyco/opencode, MIT License.（会话管理、工具目录 `.opencode/tools/`、`/session/:id/abort` 等原语出处；本项目二次开发四处见 3.3）

[14] QuantumNous. *new-api: 开源 AI 网关与资产分发系统*. github.com/QuantumNous/new-api, AGPL-3.0 License.（配额权威源与消费日志所在；嵌合边界见 3.3、许可声明见 3.4）

[15] OSideMedia. *higgsfield-ai-prompt-skill: Higgsfield「Hell Grind」长片流水线与 Seedance 工业级提示词规则库*. GitHub 仓库, MIT License.（镜头六要素、情绪具象化等配方的吸收来源；改写边界见 3.4）

[16] n8n GmbH. *n8n: Workflow Automation Platform*. n8n.io（fair-code 许可）.（表 11「流程编排工具」类对比依据）

[17] LangGenius. *Dify: Open-Source LLM Application Platform*. github.com/langgenius/dify（Apache-2.0 附加条款）.（同上）

[18] Comfy Org. *ComfyUI: The Most Powerful and Modular Diffusion GUI*. github.com/Comfy-Org/ComfyUI, GPL-3.0；其面向智能体的官方接口见 docs.comfy.org — Agent Tools（Comfy MCP、Comfy CLI）, 2026.（表 4／表 11「节点式生成工具」类对比依据：节点生态规模、模板市场与智能体接口现状俱据其官方站点实查）

[19] Manus AI. *Manus: Hand Off Complex Tasks*. manus.im, 2025.（表 11「对话式智能体」类对比依据）

[20] OpenAI. *API Reference — Images, Videos, Batches*. OpenAI Platform Documentation, 2026.

[21] Vercel Inc. *Next.js Documentation — App Router, Route Handlers, Rewrites*. 2026.

[22] PostgreSQL Global Development Group. *SELECT — The Locking Clause (FOR UPDATE SKIP LOCKED)*. PostgreSQL 16 Documentation, 2023.

[23] W3C. *Content Security Policy Level 3*. W3C Working Draft, 2023.